

Chapter 1

Introduction to Generative AI and LLMs

Overview of Generative AI, Large Language Models, evolution of NLP, transformer architecture (conceptual), applications of LLMs
introduction to prompting, tools (ChatGPT, Gemini, Claude)



ChatGPT

Helps write, explain,
solve and create.



Gemini

Answers questions,
helps with ideas,
planning and more.



Google Maps

Finds the best
routes in real time,
avoids traffic,
saves time.



**AI is already
part of our
daily lives.**



Predictive Text



Spam Detection



Music
Recommendations



Face Unlock



Language
Translation



Smart Home
Devices



What to Watch
Next

WELCOME TO THE AGE OF AI

AI is all around us!



Most people use AI
every day without realizing it.



Instagram
Shows posts
you'll
reels you'll
Learns what
like and su
more.



YouTube
Suggests
you'll enjoy
you'll enj
Learns fro
you watch



Online Shopping
Recommends
products you
might like.
Makes shopping
smarter.

IMAGINE WAKING UP TOMORROW AND ALL AI SYSTEMS SUDDENLY STOP WORKING.



WHAT WOULD HAPPEN?

TRANSPORTATION

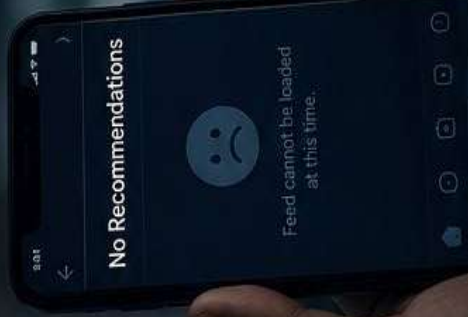
-  No Google Maps navigation
-  No traffic prediction
-  No ride-sharing optimization



 Longer travel times, missed appointments, system-wide delays.

SOCIAL MEDIA

-  No personalized feed
-  No content recommendations
-  No targeted advertisements



 Irrelevant content, wasted time, less engagement.

BANKING

 Transaction Verification Failed
Unable to process your request at this time.



 Higher risk of delays in customer service.



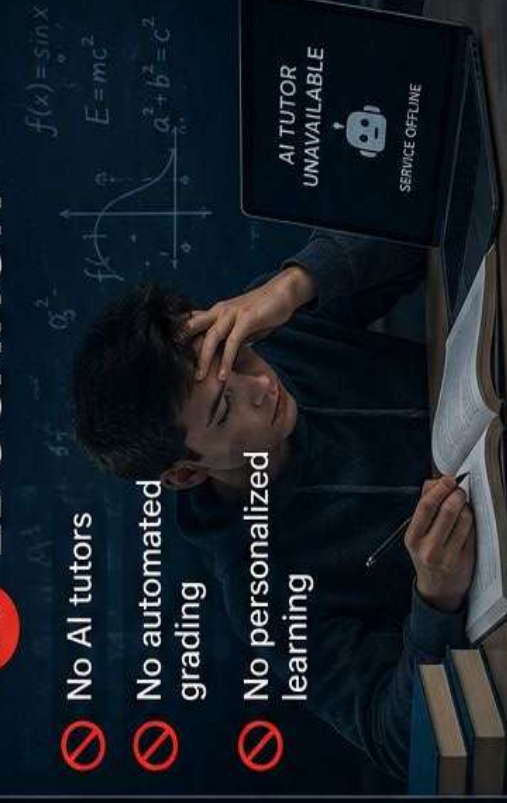
AI IS POWERING THE WORLD BEHIND THE SCENES
WITHOUT IT, EVERYTHING SLOWS DOWN.

IMAGINE WAKING UP TOMORROW AND ALL AI SYSTEMS SUDDENLY STOP WORKING.

WHAT WOULD HAPPEN?

EDUCATION

-  No AI tutors
-  No automated grading
-  No personalized learning



Slower learning.
More manual work for
teachers and students.



HEALTHCARE

-  No AI-assisted diagnosis
-  Slower medical image analysis



Delayed diagnoses.
Higher workload for
healthcare professionals.



E-COMMERCE

-  No product recommendations
-  Poor customer support



Harder to find
products.
for customers.



**THE WORLD WOULD STILL FUNCTION,
BUT MANY SERVICES WOULD BECOME SLOWER,
LESS PERSONALIZED, AND LESS EFFICIENT.**



People often define AI
as machines becoming intelligent.

But what exactly is intelligence?



When we say a person is intelligent, what do we mean?



Intelligent people can:



Learn

Acquire knowledge
from experiences.



Reason

Think logically.



Solve Problems

Find solutions to
challenges.



Adapt

Adjust to new
situations.



Communicate

Express and
understand ideas.



Human intelligence
is not simply about
memorizing facts.



It is about **understanding, reasoning, adapting, and creating.**

Human Intelligence vs Artificial Intelligence



Humans and AI can both solve problems, but they do so in very different ways



HUMAN INTELLIGENCE



HUMANS LEARN THROUGH:



Experiences



Emotions



Observation



Social interactions

HUMANS POSSESS:



Creativity



Intuition



Common sense



Ethical judgment



VS



ARTIFICIAL INTELLIGENCE



AI POSSESSES:



Speed



Scalability



Pattern recognition

AI LACKS:



Emotions



Common sense



True understanding



Human-like judgment



Humans bring meaning, values, and wisdom.



AI brings speed, accuracy, and data-driven insights.

The future is not about humans vs AI.

It's about humans and AI working together.

What is Artificial Intelligence?

Artificial Intelligence refers to systems that can perform tasks that typically require **human intelligence**.



THESE TASKS INCLUDE:

Understanding Language



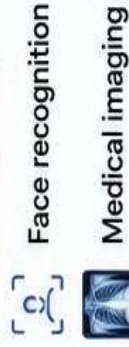
Examples:



Recognizing Images



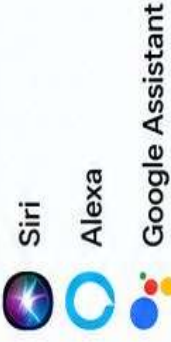
Examples:



Understanding Speech



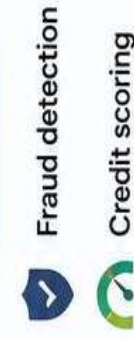
Examples:



Decision Making



Examples:



Formal Definition

Artificial Intelligence is the field of computer science focused on creating systems capable of performing tasks that normally require human intelligence.

Why AI Became Important



Artificial Intelligence is not new.
Researchers have been studying AI since the 1950s.



For decades AI progressed slowly.



Then suddenly the world experienced an **AI explosion**.



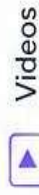
1

MASSIVE DATA

Every day people generate:



Photos



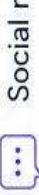
Videos



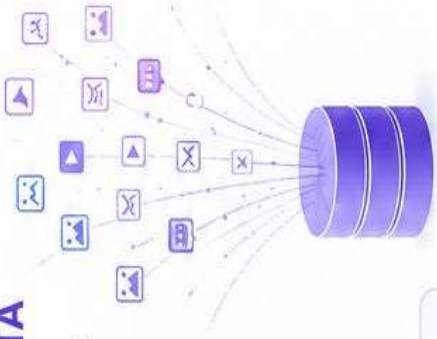
Emails



Documents



Social media posts



This data became **training material** for AI.

THREE MAJOR REASONS:

2

POWERFUL COMPUTERS

Modern GPUs can perform trillions of calculations per second.



Without GPUs, modern AI would **not exist**.

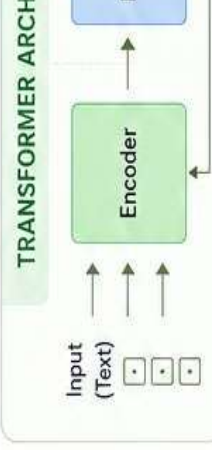
3

BETTER ALGORITHMS

Researchers developed more powerful learning techniques.

Most importantly:

The Transformer architecture revolutionized AI.



THESE THREE FACTORS TOGETHER TRIGGERED TODAY'S AI REVOLUTION.



Massive Data



Powerful Computers



Better Algorithms



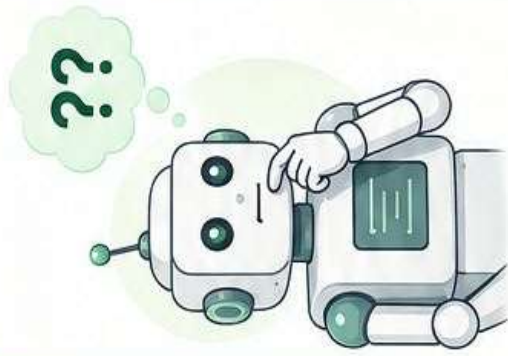
ARTIFICIAL INTELLIGENCE

A JOURNEY FROM IDEA TO IMPACT

1950s

BIRTH OF AI

Researchers asked:
Can machines think?



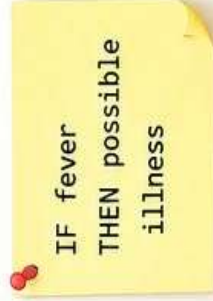
Idea

Can machines think?

1980s

EXPERT SYSTEMS

Machines followed predefined rules.



Problem:
No learning capability.



Rules

Machines follow instructions.

1997

IBM DEEP BLUE

defeated world chess champion
Garry Kasparov.



This shocked the world.
For the first time, a machine outperformed a human in a complex intellectual task.



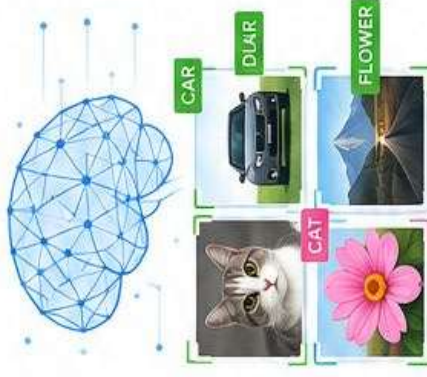
Win

Machines can beat humans.

2012

DEEP LEARNING REVOLUTION

AI began achieving remarkable performance in image recognition.



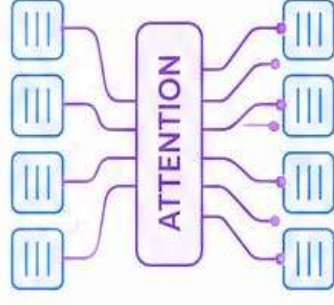
See

Machines learn from data.

2017

TRANSFORMER ARCHITECTURE

A breakthrough that eventually enabled ChatGPT and modern AI systems.



Understand

Machines understand context and language.

AI, Machine Learning, Deep Learning, and Generative AI



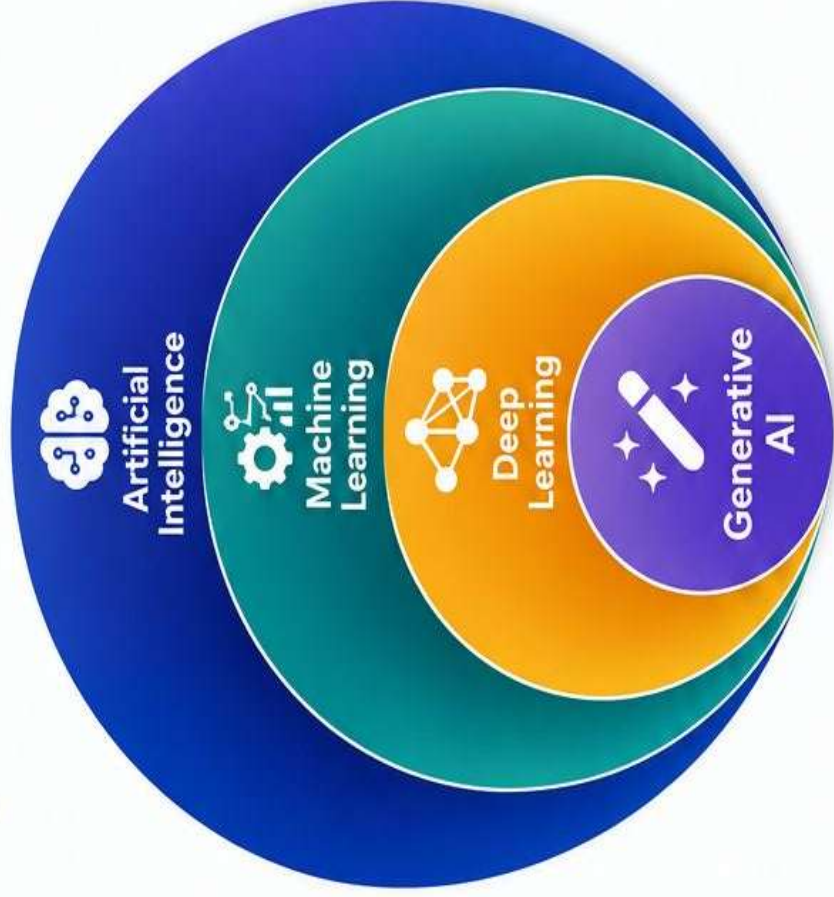
Many people use these terms interchangeably.



However, they are different.



Think of them as **nested layers**.



Artificial
Intelligence

Machine
Learning

Deep
Learning

Generative
AI



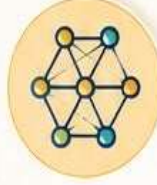
Artificial Intelligence

The broad field.



Machine Learning

Machines learn
from data.



Deep Learning

Machines learn using
neural networks.

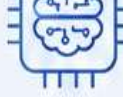
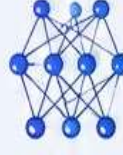


Generative AI

Machines create
new content.



This hierarchy is crucial to understand
before studying **Prompt Engineering**.



How Machines Learn



Imagine **teaching a child** to recognize **cats**.

You show:

10 cat images



100 cat images



10,000 cat images



Eventually the child **learns patterns**.

The child notices

Ears



Fur



AI learns similar



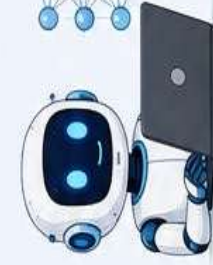
The process:

1. Data



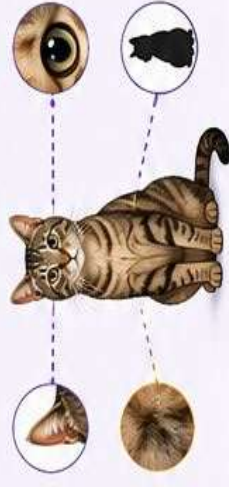
Many examples
(images, text, etc.)

2. Training



The AI model studies
the data

3. Pattern Recognition



AI identifies important
patterns and features

4. Prediction



AI uses previous
data to make accurate
predictions



AI **does not memorize everything**.



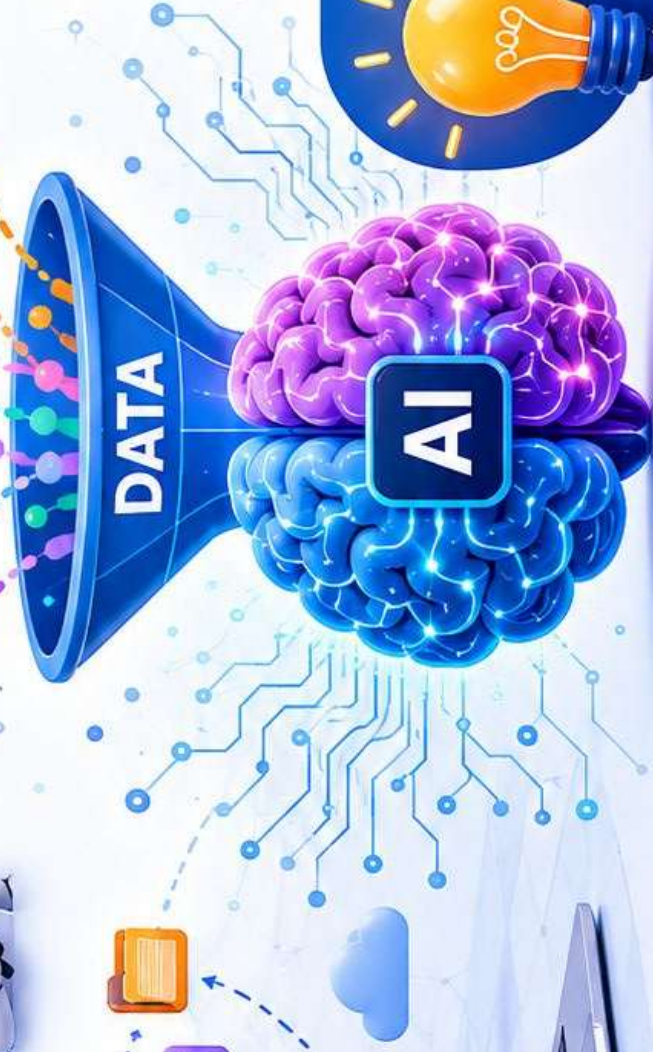
AI **discovers patterns**.

Data: The Fuel Behind AI

Examples of AI Training Data



An AI model is only as good as its data.



What Happens



Limited

AI performance suffers.



Biased

AI becomes biased.



Poor Quality

AI produces poor outputs.

Key Insights

Data teaches AI what the world is like.



What is

MACHINE LEARNING?

+



Instead of writing every rule manually,
we allow computers to **learn patterns** from examples.

Ex

1

Traditional Programming



Input + Rules → Output



Humans write
the rules

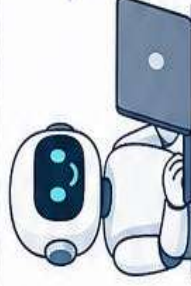
VS

2

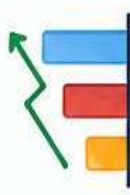
Machine Learning



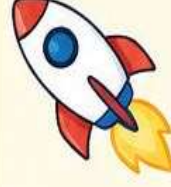
Input + Output Examples →
Rules Learned Automatically



The machine learns
the rules



Machine Learning represented a
major shift in computing.



Machines started **learning**
simply following instructions

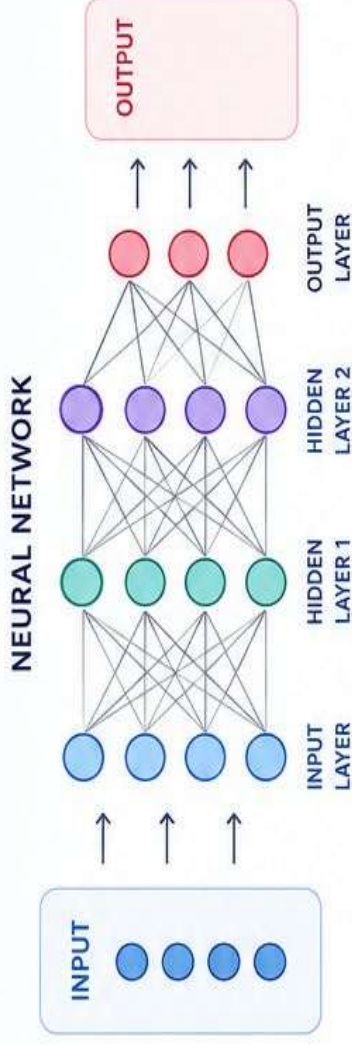


DEEP LEARNING: — INSPIRED BY THE HUMAN BRAIN —

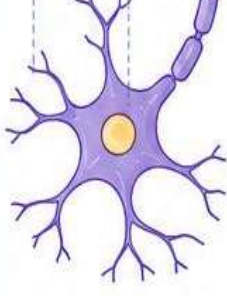
Deep Learning uses Neural Networks.

A neural network consists of interconnected nodes that process information.

Inspired by biological neurons.



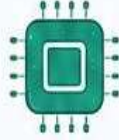
INSPIRED BY BIOLOGY



DEEP LEARNING BECAME SUCCESSFUL BECAUSE:



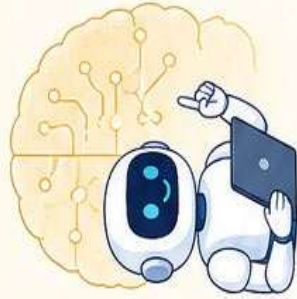
More data
became available



Computing power
increased



Algorithms
improved



DEEP LEARNING LAID THE FOUNDATION FOR GENERATIVE AI.



Image
Generation



Text
Generation



Music
Generation



Video
Generation



Face Recognition
Identifies or verifies individuals from photos



Voice Recognition
Converts speech into text and understands context



Medical Diagnosis
Detects diseases and suggests treatments for doctors with better accuracy



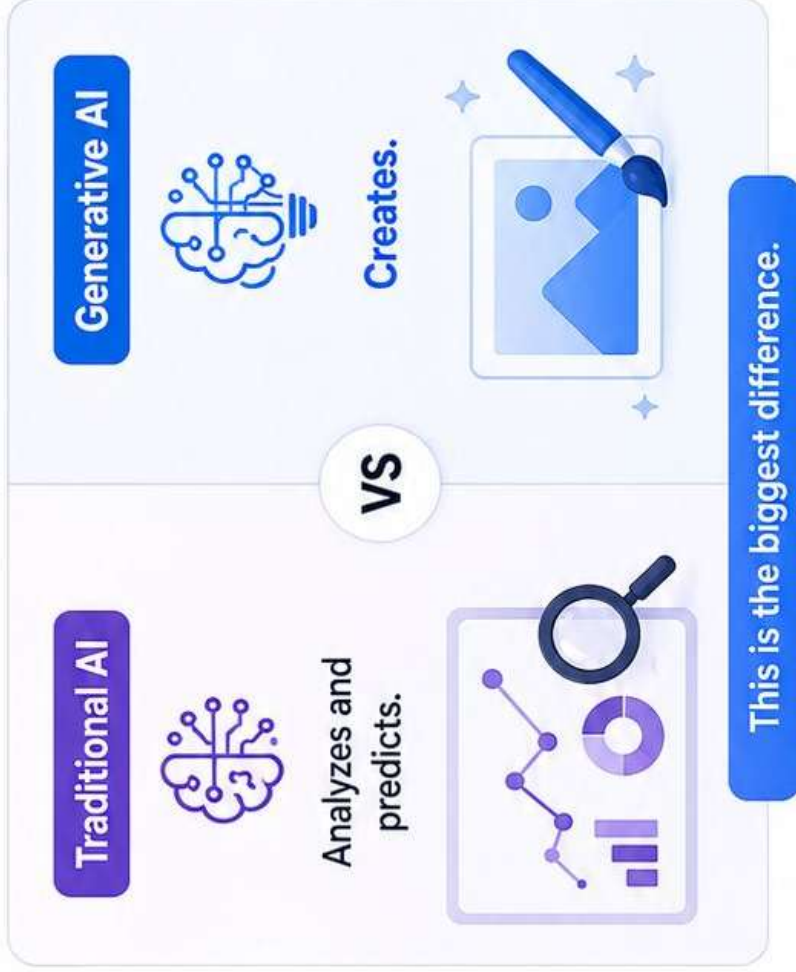
Autonomous Systems
Enables self-driving cars to perceive and navigate environments



Deep Learning mimics the way our brain learns from experience, enabling machines to solve complex real-world problems.



Enter Generative AI



This capability fundamentally changed
how humans interact with computers.



For the
computer
content

Generative AI can create:



Essays

Well-structured
written content on
any topic.



Stories

Creative stories
and narratives
in any style.



Images

Stunning visuals
from simple
text prompts.



Why Generative AI Feels Magical



When people use ChatGPT for the first time, they often feel like they are talking to an **intelligent human**.



Why?

Because it can:



Understand questions



Generate responses



Explain concepts



Write code



Create stories

Wow! That was so helpful!

I'm here to help!



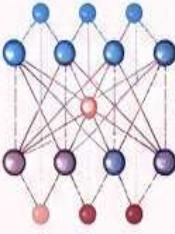
However, behind the scenes there is no magic.



Mathematics



Statistics



Neural Networks



Massive Training Data

=



Your Input (Question)



Pattern Prediction (Trained Model)



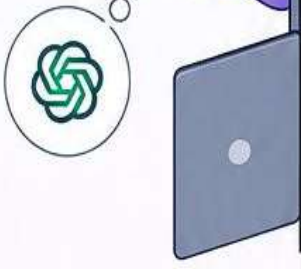
The illusion of intelligence comes from extremely sophisticated pattern prediction

THE MOST IMPORTANT QUESTION

— If everyone has access to the same AI... —



WHY DO SOME PEOPLE
GET **AMAZING** RESULTS
WHILE OTHERS GET
POOR RESULTS?



This question leads
directly to
Prompt Engineering.

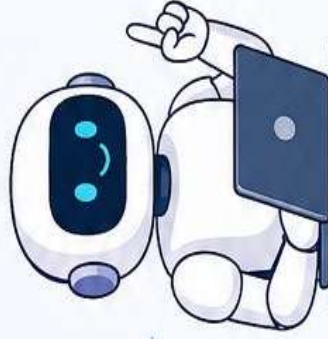


The difference is not the AI.
The difference is how you
communicate with the AI.



And from here you
naturally into:

PROMPT ENGINEERING: THE LANGUAGE OF AI



AI is powerful,
but it's **your prompt**
that unlocks its true
potential.



A well-crafted prompt
gives clear direction.



Clear direction
leads to better output.



Better output
creates more value.

THE BETTER THE PROMPT, THE BETTER THE RESULT

VAGUE PROMPT (Poor Result)

"Write about marketing."



- Generic
- Too broad
- Not useful

CLEAR, WELL-CRAFTED PROMPT (Amazing Result)

"Write a detailed marketing strategy for small businesses in 2024, focusing on digital marketing and actionable tips."



- Specific
- Relevant
- Actionable
- High quality



Master the language. Get better results. **Create** more impact.



INTRODUCTION TO

LARGE LANGUAGE MODELS (LLMs)



WHAT IS A LARGE LANGUAGE MODEL?

A Large Language Model (LLM) is an Artificial Intelligence system trained on **massive amounts of text data** to **understand, process, and generate** human language.



Answer questions



Write essays



Summarize documents



Generate code



Translate languages

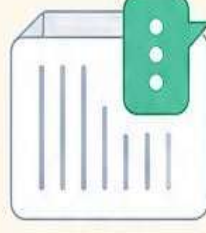
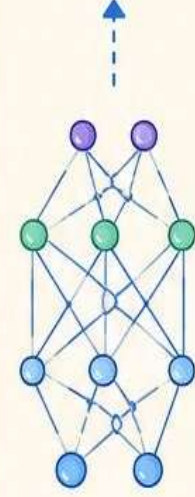


Create stories



Assist with research

THESE MODELS CAN:

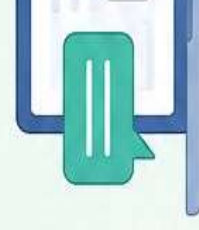


Unlike traditional software, LLMs are **programmed** with every possible response. Instead, they **learn language** from enormous datasets and **responds** dynamically.



KEY MESSAGE

LLMs don't just find information.
They understand language and generate new content that feels natural and helpful.



EXAMPLES



ChatGPT



Gemini



Claude



Microsoft



Llama

Why Do We Need Large Language Models?



Computers traditionally struggle with human language because language is:



Ambiguous



Context-dependent



Dynamic

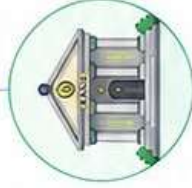


Full of nuances

Consider the word:

“Bank”

It could mean:



Financial institution



River bank



Humans understand the meaning from context.



Machines historically struggled with this.

Why Traditional AI



Traditional AI works with structured data and structured tasks.



But human language is **complex**, and computers struggle to process it.



LLMs are trained on **trillions of tokens** of **text data** to learn patterns and relationships.



This allows them to generate text that is **coherent** and **similar to human** output.



Large Language Models are developed to bridge the gap between human language and machine processing.



LLMs process vast amounts of text, learn context, capture nuances, and help machines understand the richness of human language.



Key Insight

The goal of LLMs is to understand and generate human language in a way that is coherent and similar to human output.

Evolution of Language AI



Understanding LLMs requires understanding **how language processing evolved**.

Phase 1

Rule-Based Systems

Early systems used manually written rules.

IF user says "Hello"
THEN respond "Hi"



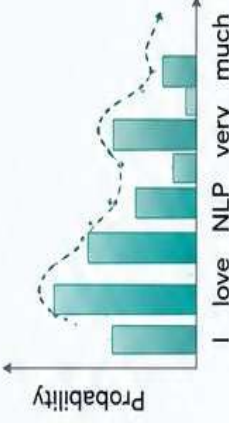
Hi!

Problem:
Could not handle complex conversations.

Phase 2

Statistical NLP

Systems learned word probabilities.



Probability

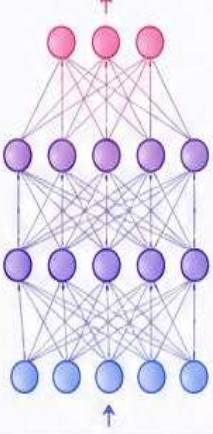
I love NLP very much

Improved performance but lacked deep understanding.

Phase 3

Deep Learning

Neural networks improved language understanding.

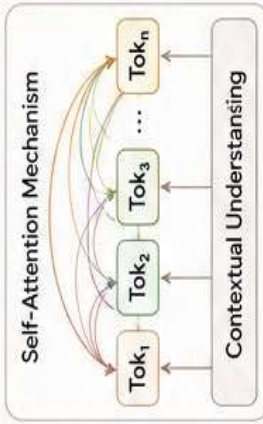


The system can capture complex patterns in language.

Phase 4

Transformers

Introduced in 2017.
Became the foundation of modern LLMs.



Self-Attention Mechanism

Contextual Understanding

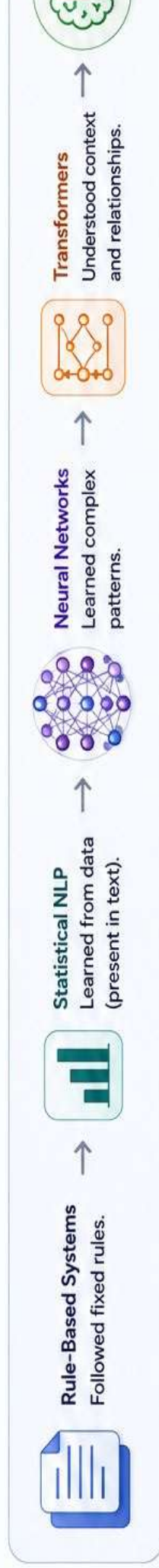
Understands context better, even in long sequences.

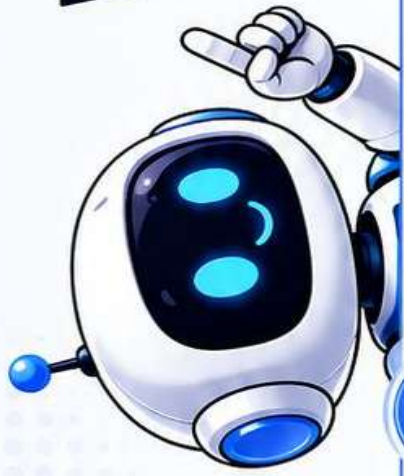
1950s – 1980s

1990s – 2000s

2010 – 2016

2017





What Makes an LLM “Large”?

The word “Large” refers to two major aspects.

1

Large Amount of Data

LLMs are trained on a massive amount of data from many sources.



Books



Articles



Research papers



Websites



Documentation



Conversations



Source code



2

Large Number of Parameters

Parameters are internal values learned during training.



Think of parameters as **knowledge connections** inside the model.



Examples:

GPT-2



1.5

Billion
Parameters

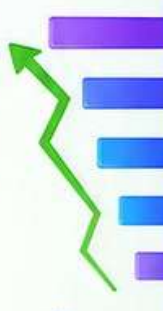
GPT-3



175

Billion
Parameters

Modern Models



Hundreds of billions or even **trillions** of parameters.

HOW DOES AN LLM LEARN?

Imagine giving a student millions of books.



Over time the student learns:



Vocabulary



Writing styles



Grammar

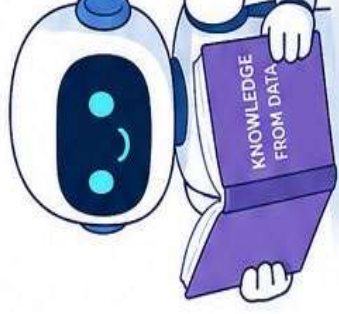


Relationships between concepts



Facts

LLMs learn similarly.



TRAINING PROCESS

1

Read massive text datasets



The model reads billions of words from books, websites, articles, code, and more.

2

Predict missing words



Given a sentence, the model tries to predict what comes next.

3

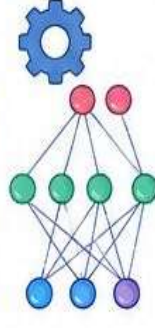
Compare predictions with actual words



The model checks if its prediction is correct.

4

Adjust internal parameters



It adjusts millions (or billions) of parameters to reduce errors.

5

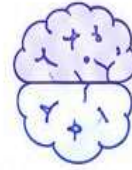


The model



Through this process, the model gradually learns language patterns.

! IMPORTANT



The model does not memorize every sentence. It would be impossible!



It learns statistical relationships between words and concepts. That's how it can generate, answer, explain, and create new content.



It understands patterns, not

Next Word Prediction: The Core Idea



At their core, LLMs are trained to
predict the next most likely word.

EXAMPLE 1

Sentence:

"The capital of India is ____"



Likely prediction:

"New Delhi"

EXAMPLE 2

Sentence:

"The sun rises in the ____"

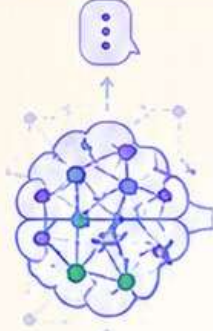


Prediction:

"east"



By repeating this prediction process
billions of times, the model learns
language structure, grammar,
facts, reasoning patterns, and
much more.



IMPORTANT CONCEPT

Complex intelligence emerges from
a surprisingly simple learning objective:

Predicting the next v

HOW IT WORKS

The



98%

capital



95%

of



97%

India



96%

is



93%

The



99%

sun



98%

rises



97%

in



96%

the



94%

Transformer Architecture: The Breakthrough



Modern LLMs are based on the Transformer architecture.



Introduced in 2017 through the paper: *"Attention Is All You Need"*



The Transformer introduced a powerful concept:

Attention Mechanism

Attention allows the model to focus on the most relevant words in a sentence.

Example:

"The animal didn't cross the road because **it** was too tired."

When the model processes "it",



The model pays more attention to relevant words:



High Attention

Low Attention

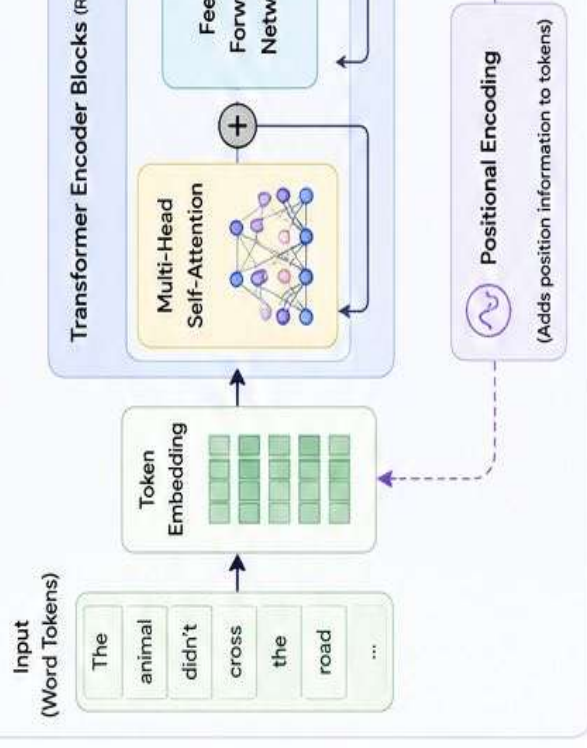


The model learns that "it" refers to the animal.



This ability dramatically improved language understanding.

The Transformer Architecture (Simplified)



Why Transformers

Without Transformers



ChatGPT would not exist

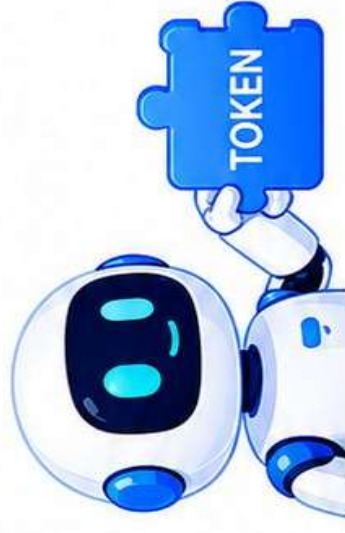


Gemini would not exist



Transformers changed the game. They made **human-like understanding** possible.

Understanding Tokens



LLMs do not process text exactly like humans.
Instead they process: **Tokens**

A token may be:

Aa

A word



Part of a word

,

A punctuation mark

Example:

“Artificial Intelligence”

Could become:

Artificial

|

Intelligence

or

Artifi

|

cial

|

Intelligence

depending on tokenization.



Tokens are the **basic units** processed by LLMs.

Why This



Model limits are
measured in
rather than

Example

Text (Words): Artificial Intelligence is transforming the world.

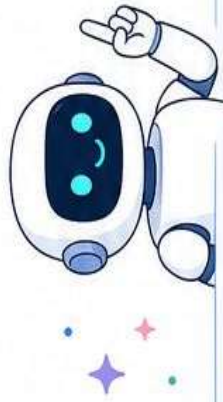
As Tokens (Example):

Artificial

Intelligence

is

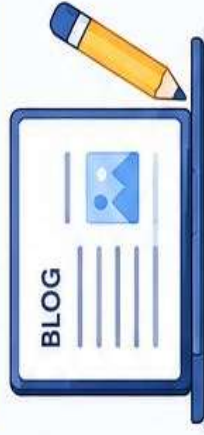
transforming



WHAT CAN LLMs DO?

LLMs are incredibly versatile and can understand and generate human-like text across a **wide range** of tasks and industries.

1 CONTENT CREATION



- ✓ Blogs
- ✓ Articles
- ✓ Emails



2 EDUCATION



- ✓ Tutoring
- ✓ Question generation
- ✓ Explanations



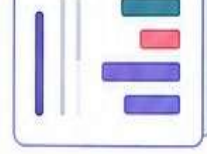
3 PROGRAMMING



- ✓ Code generation
- ✓ Debugging



4 BUSINESS



- ✓ Reports
- ✓ Documenta

5 RESEARCH



- ✓ Summaries
- ✓ Literature reviews



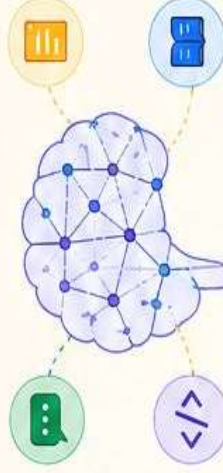
6 CUSTOMER SUPPORT



- ✓ Chatbots
- ✓ Virtual assistants

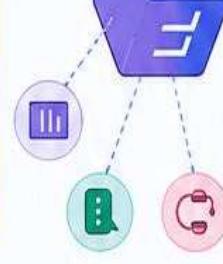


WHY SO VERSATILE?



The model has learned **general language patterns**, not task-specific rules.

SAME MODEL



One powerful model
Countless real-world applications



The same model can perform many tasks because it has learned general language patterns.

This is what makes LLMs so powerful and adaptable!

What LLMs Cannot Do Perfectly

Despite their capabilities, LLMs have important limitations.



Key Message:

**LLMs are powerful assistants,
not infallible experts.**



Why Prompt Engineering Becomes Important



If the same LLM is available to everyone:

Why do different people get different results?



The answer is:
Prompts

The quality of output depends heavily on:



Instructions



Context



Examples



Constraints



Clarity



Same model.

Different prompt.

Different output quality.

Example:



Poor Prompt:
"Explain AI"



Artificial Intelligence (AI) is the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions.



Artificial Intelligence (AI) is the simulation of human intelligence in machines that are normally re

Examples:

- Voice assistant
- Recommendation system like Netflix
- Self-driving car

In (p



Output Quality:
Basic and generic



Output Quality:
Clear, detailed



Better Prompt:
"Explain AI to students in a real-world context"



Transition

Understanding LLMs helps us understand why **Prompt Engineering** has become one of the most valuable AI skills today.

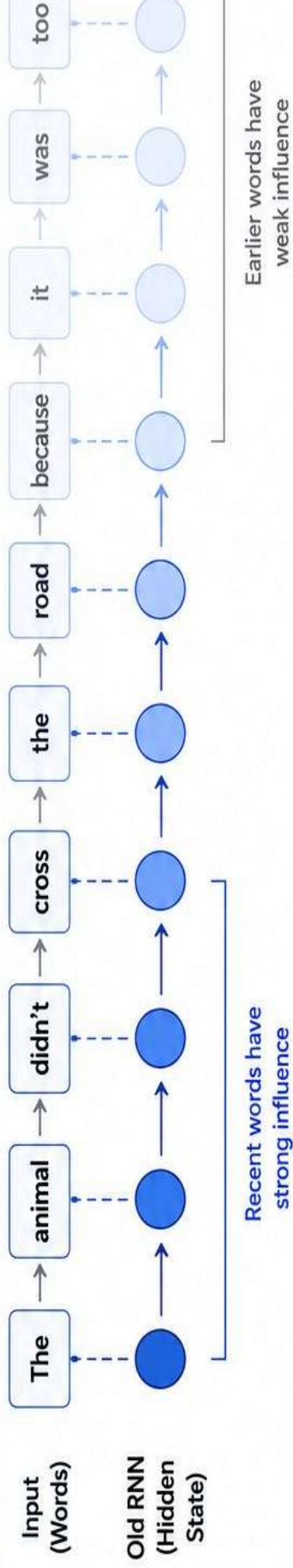


The Problem Before Transformers

Example: Long Sentence

“The animal didn’t cross the road because it was too tired.”

How Old Neural Networks (RNNs) Process Text



As the sentence gets longer, the model struggles to remember early words and their im

1

Sequential Processing



Words are processed one by one from left to right.

2

Slow Training



RNNs are inherently sequential, making training slower and harder to parallelize.

3

Long-Term Memory Problem



The model has difficulty remembering information from far back in the sequence.

4



Key Takeaway:

Old neural networks (RNNs) process text sequentially and suffer from context loss, making it hard to capture long-distance relationships in language.

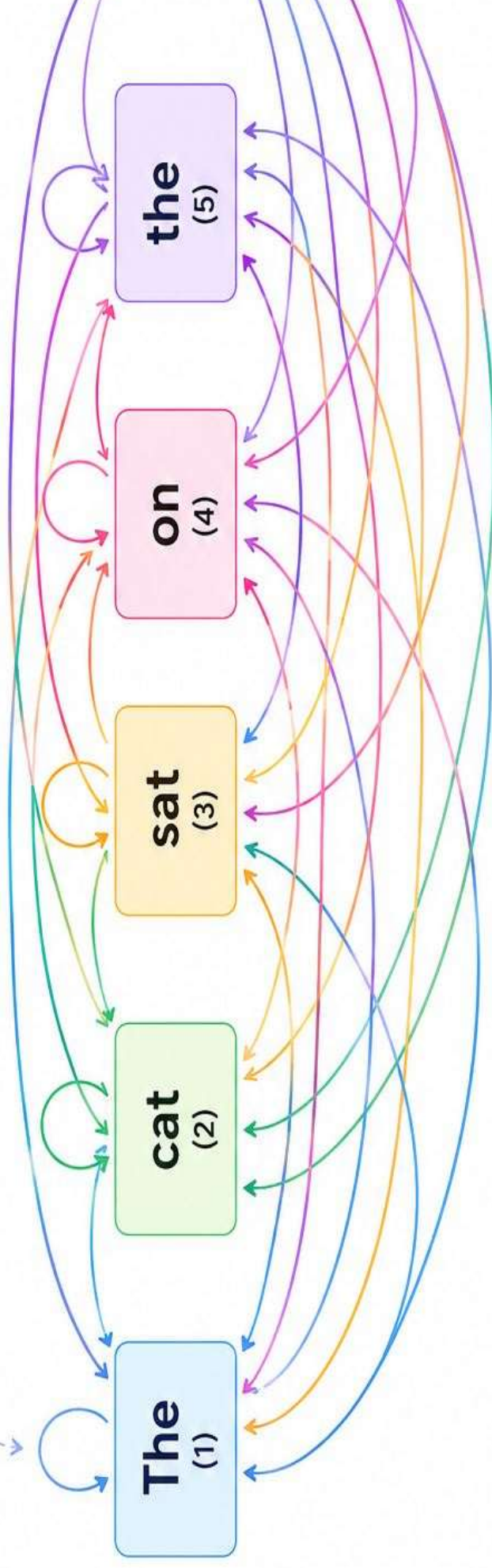


SELF-ATTENTION



SENTENCE: "The **cat** **sat** **on** the **mat**."

EACH WORD LOOKS AT ALL OTHER WORDS



EVERY WORD LEARNS FROM EVERY OTHER WORD.

This helps the model understand the context and relationships between all words in the sentence.



Direct at
(word to)



Context
(built from)



MULTI-HEAD ATTENTION

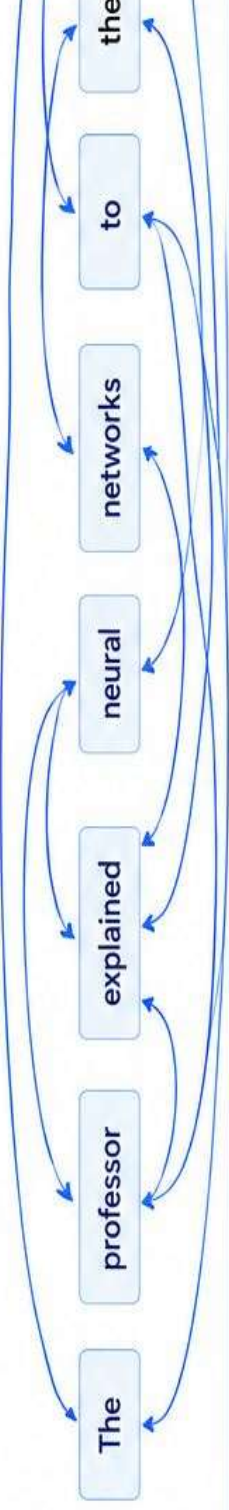


SENTENCE: “The professor explained neural networks to the



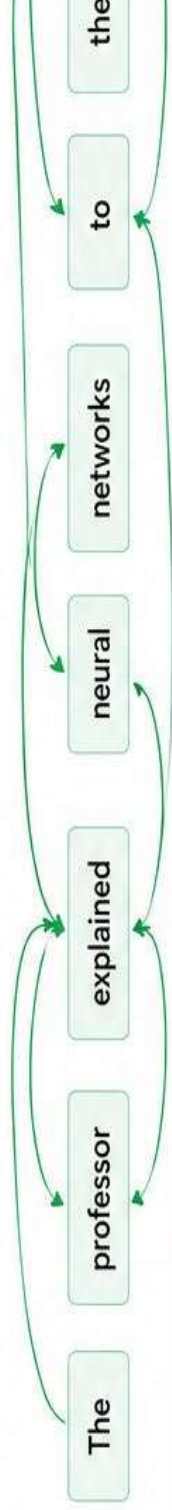
HEAD 1

FOCUS: GRAMMAR
Looks at grammatical structure and parts of speech.



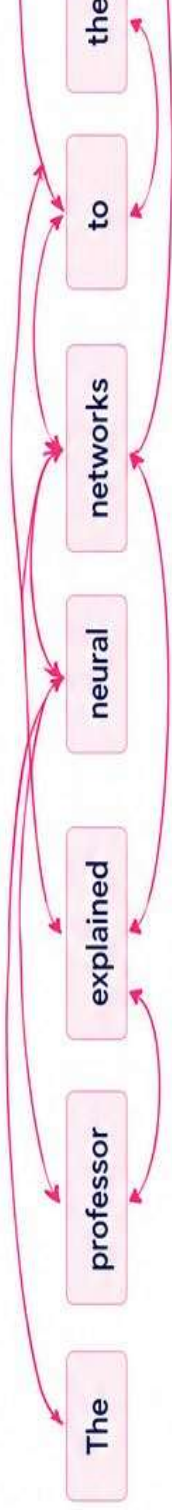
HEAD 2

FOCUS: SUBJECT-ACTION
Focuses on who is doing what to whom.



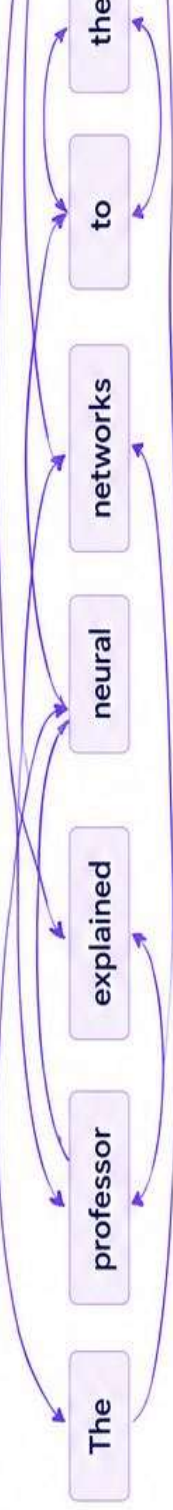
HEAD 3

FOCUS: MEANING
Captures semantic relationships between important words.



HEAD 4

FOCUS: CONTEXT
Understands the overall context and flow of the sentence.



THE MODEL LOOKS AT THE SENTENCE FROM MULTIPLE PERSPECTIVES SIMULTANEOUSLY.

Each head attends to different aspects of the sentence. Together, they help the model understand language deeply.



Multi-head attention model to capture by combining the many attention heads

Why Positional Encoding is Needed

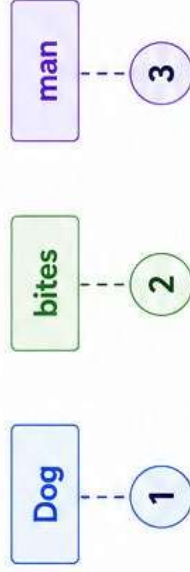
Transformers process all words at once (in parallel). To understand order, we add **position information** to each word.

Sentence 1

“Dog bites man”

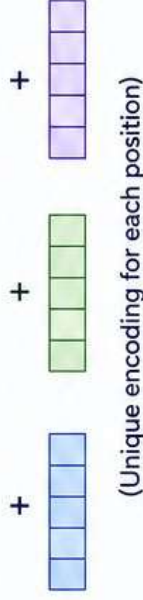
Word Position

Each word has a position in the sequence.



Positional Encoding

Position information is added to each word.



Meaning:

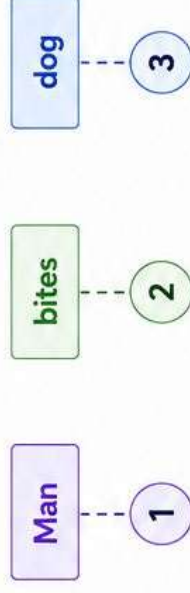
A dog is doing the biting to a **man**.

Sentence 2

“Man bites dog”

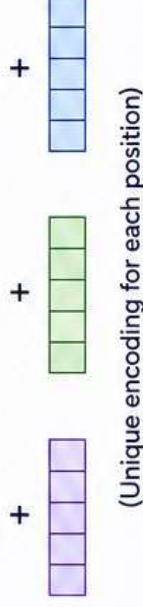
Word Position

Each word has a position in the sequence.



Positional Encoding

Position information is added to each word.



Meaning:

A **man** is doing the biting to a **dog**.

Why Position Matters



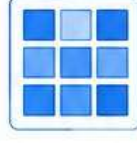
The same words in different order mean **different meanings**.

Sequence Information



Word order changes the meaning of the sentence.

Positional Encoding



By adding positional information to each word, the model can understand the position of words in the sequence.

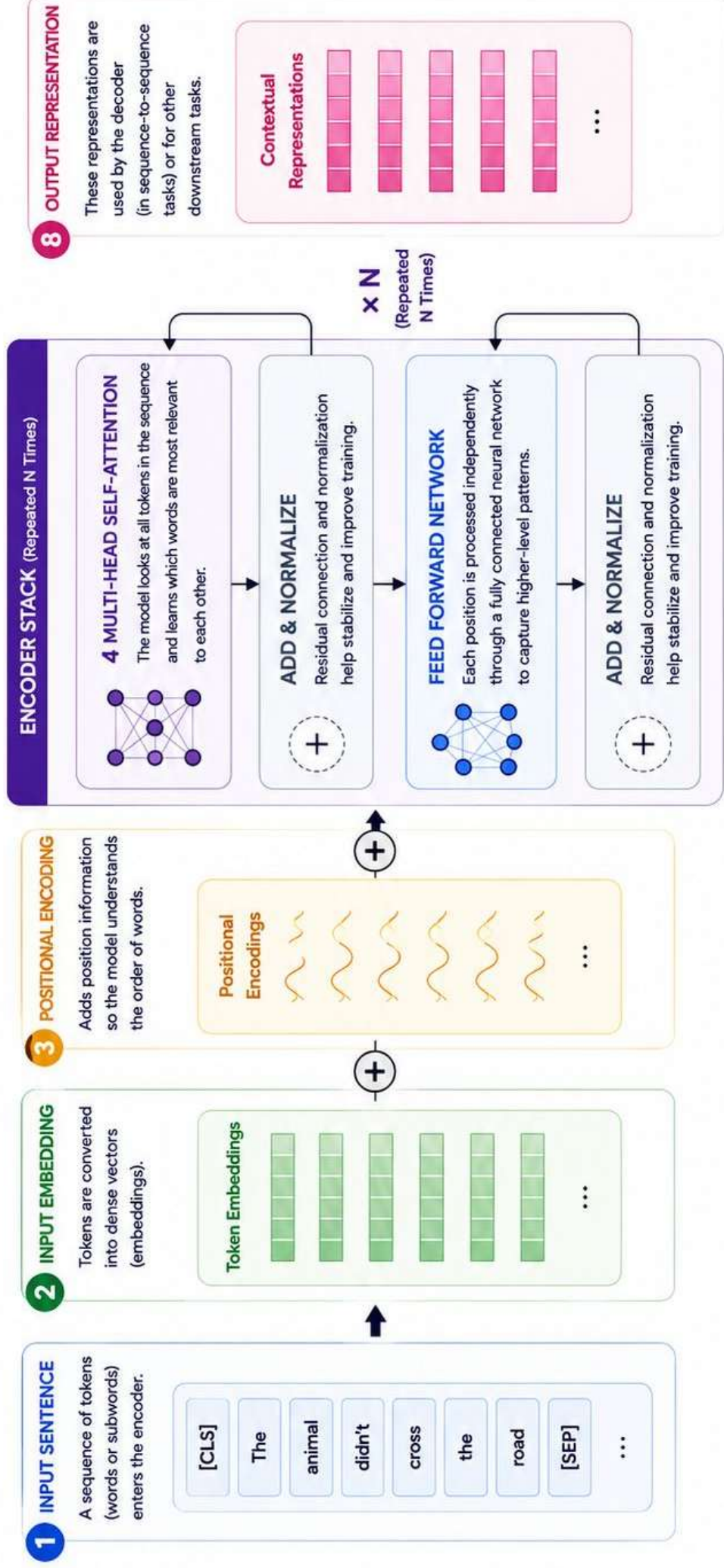


Key Takeaway:

Without positional encoding, a model would see “Dog bites man” and “Man bites dog” as the **same set of words** and might not understand the difference.

Transformer Encoder

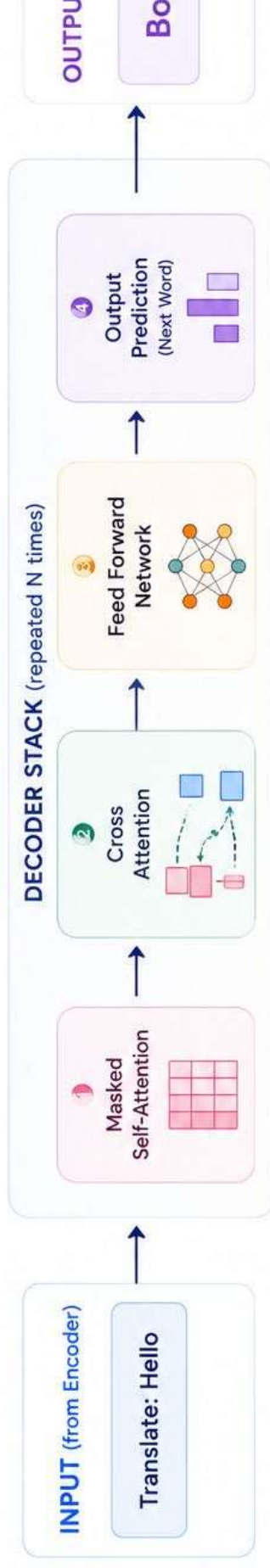
The encoder converts an input sequence into **contextual representations** that capture the meaning of the entire sequence.



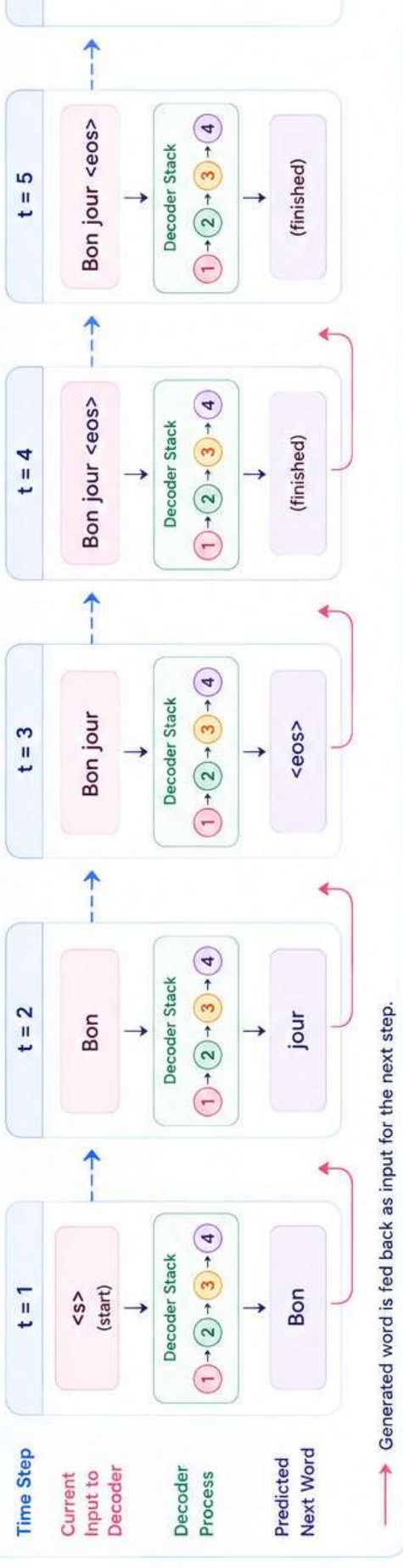
Understanding Component: The Transformer Encoder turns an input sequence into rich, context-aware representations using attention and neural networks.

TRANSFORMER DECODER

GENERATION COMPONENT



HOW THE DECODER GENERATES OUTPUT (ONE WORD AT A TIME)

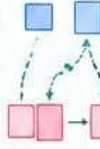


1 MASKED SELF-ATTENTION



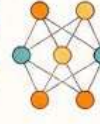
Each position can only attend to earlier positions (including itself). Prevents looking at future words.

2 CROSS ATTENTION



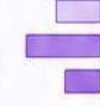
Decoder attends to the encoder output to get relevant information from the input sequence.

3 FEED FORWARD NETWORK



Applies transformations to combine the information and create rich representations.

4 OUTPUT PREDICTION



Predicts the probability distribution over the vocabulary to choose the next word.



The decoder generates the output sequence one word at a time until it produces the end token <eos>.

EXAMPLE

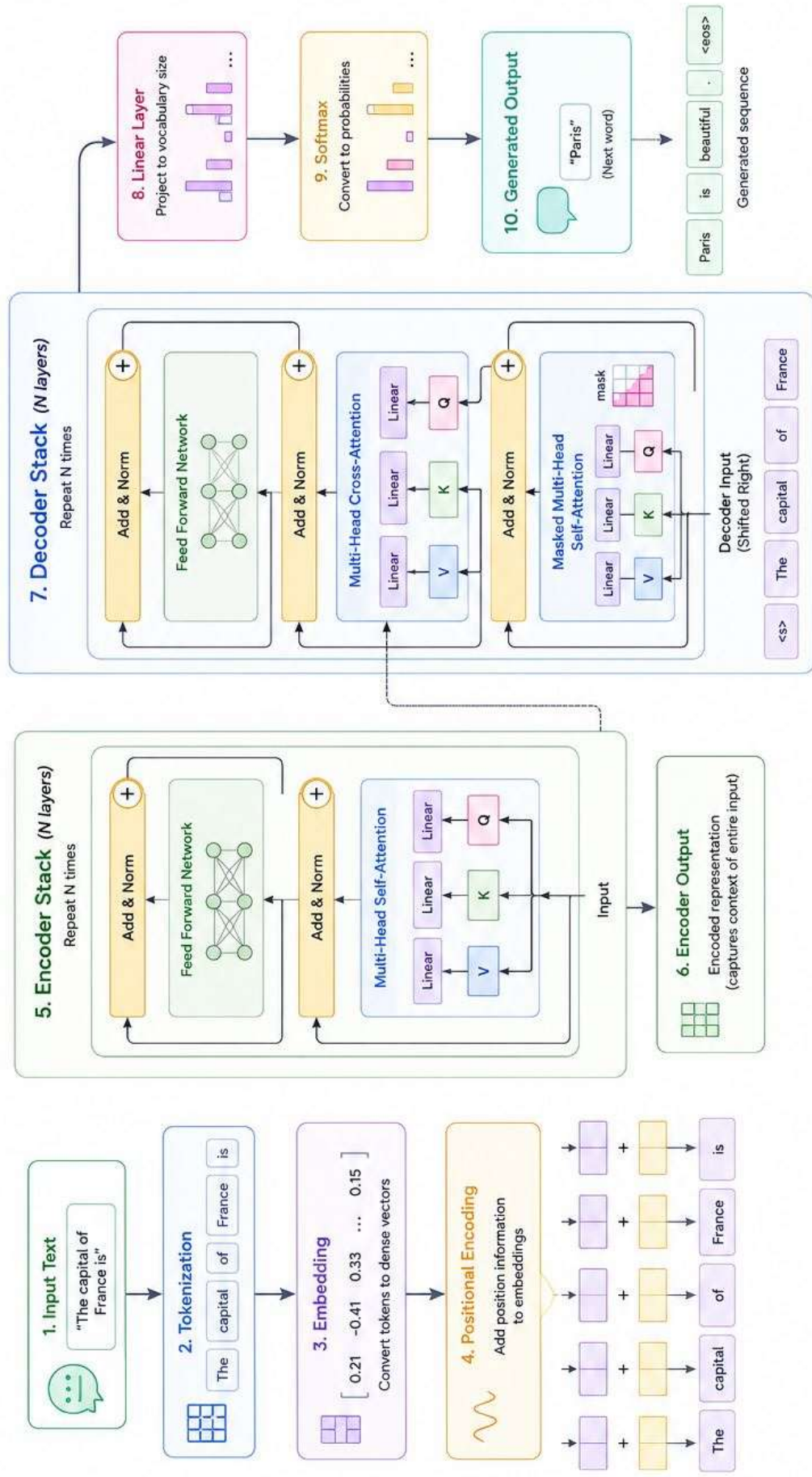
Input: Translate Hello



Output: Bonjour

Complete Transformer Architecture

The Transformer model consists of an Encoder-Decoder architecture based on Self-Attention.



How It Works



1. Input text is tokenized, embedded and combined with positional information.
2. Encoder processes the input using self-attention to capture context.
3. Decoder generates output one token at a time, attending to both previous tokens and encoder output.
4. Softmax produces probabilities over the vocabulary for the next token.



Notes

- N (number of layers) is a hyperparameter (e.g., 6, 12, 24).
- Multi-Head Attention allows the model to focus on different parts of the sequence simultaneously.
- Masked Self-Attention ensures the decoder cannot see future tokens.



Summary

The Transformer uses attention exclusively, enabling powerful and state-of-the-art performance on a range of NLP tasks.

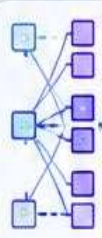


From Transformers to ChatGPT

The journey from a powerful architecture to helpful AI you can chat with.

1

Transformer Architecture

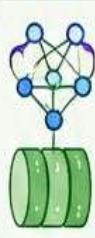


- Revolutionary neural network architecture
- Uses self-attention to understand relationships
- Processes all words in parallel



2

Large Language Models (LLMs)



- Transformers scaled up with massive amounts of data
- Learn patterns, knowledge, and reasoning abilities
- General purpose text understanding and generation



3

GPT (Generative Pre-trained Transformer)



- OpenAI's family of LLMs based on the Transformer
- Trained on vast amounts of text data
- Able to follow instructions and generate human-like text



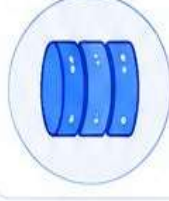
4

ChatGPT



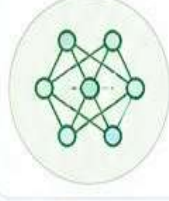
- Fine-tuned GPT model optimized for conversation
- Aligned to be helpful, safe, and honest
- Easy-to-use chat interface for everyone

Increasing Scale: Key Ingredients



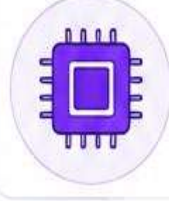
More Data

Trained on billions of words from books, articles, websites, code, and more.



More Parameters

From millions to billions to hundreds of billions of parameters.



More Computing Power

Huge compute resources enable training of very large models.

What This Enables



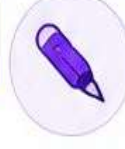
Question Answering

Answer factual and open-ended questions.



Coding

Write, debug, and explain code in many languages.



Writing

Create stories, essays, emails, and more with ease.



Summarization

Summarize long texts into concise key points.



Key Takeaway:

Transformers provided the architecture. Scale (data, parameters, compute) created intelligence. Alignment and fine-tuning made it helpful. That's how we get from Transformers to ChatGPT.

Introduction to Prompting & AI Tools

Prompting is the way we communicate with AI models. The better your prompt, the better the response.

What is Prompting?



A prompt is your input or question given to an LLM to get a response.

Good prompts are:



Clear



Specific

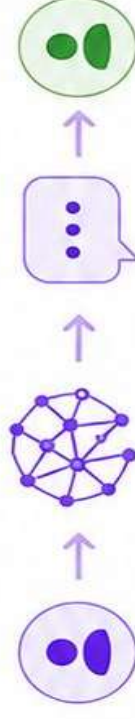


Contextual



Goal-oriented

How Prompting Works



You give a prompt

The LLM processes it

The LLM generates a response

You get a valuable output

Tips for Better Prompts



Be clear and specific



Provide context – Give background information



Specify the format – Ask for tables, steps, etc.



Iterate and refine – Improve based on the response.

Popular AI Tools



ChatGPT

by OpenAI

A versatile AI assistant for answering questions, writing content, coding help, brainstorming, and much more.

Text Generation

Coding

Analysis

Brainstorming



Gemini

by Google

A multimodal AI model that understands and generates text, images, audio, and more. Integrated with Google services.

Multimodal

Web Search

Analysis

Google Integration



Claude

by Anthropic

An AI assistant focused on safety and helpful responses for a wide range of tasks.

Summarization

Writing

Reasoning



Key Takeaway

Prompting is a skill. Practice, experiment, and refine your prompts to get the most out of AI tools.

Transformer Architecture: Complete End-to-End



Example Sentence

"I love India very much"



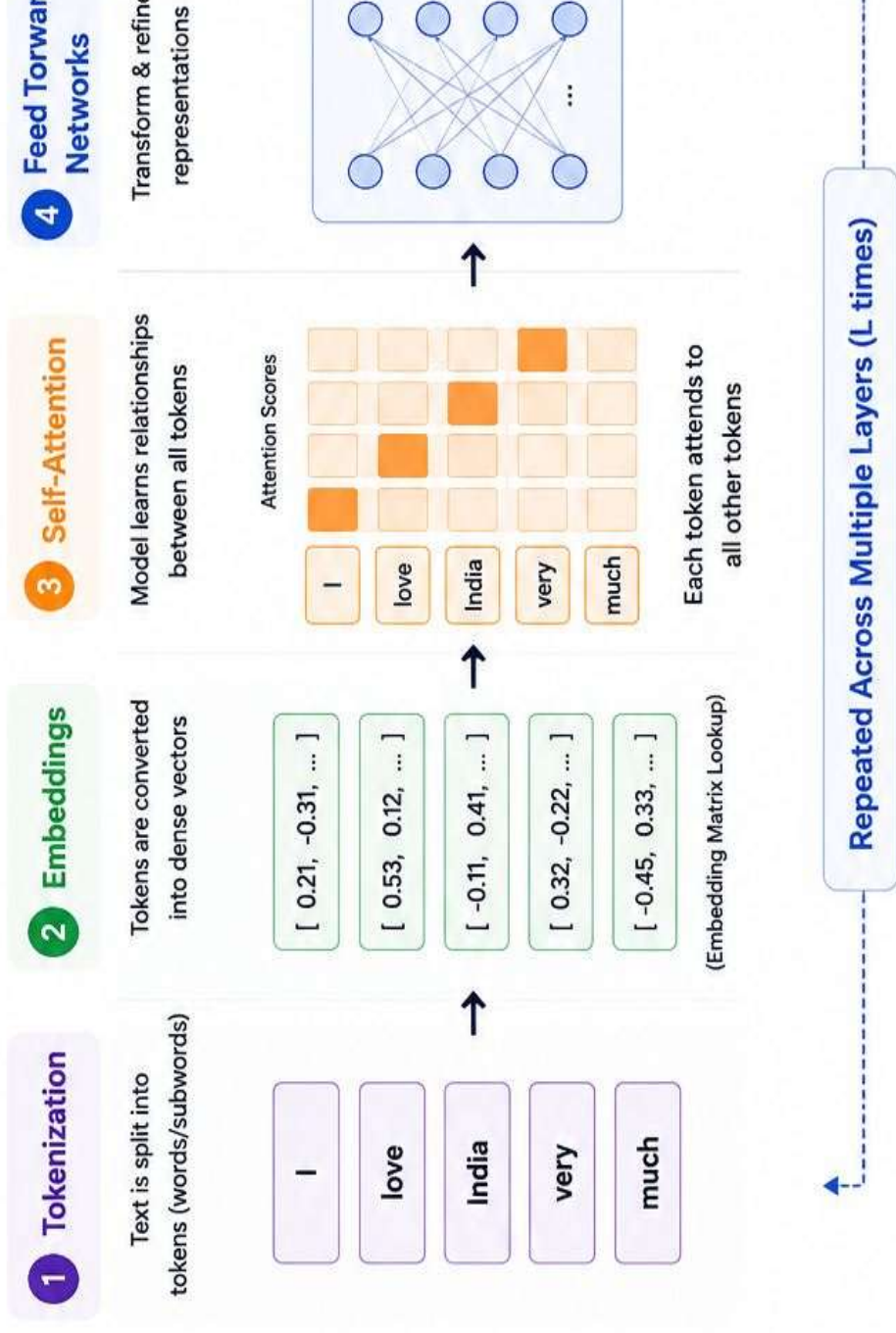
Learning Objectives

- ✓ Understand Tokenization
- ✓ Understand Embeddings
- ✓ Learn Self-Attention
- ✓ Learn Feed Forward Networks
- ✓ Understand Next Token Prediction



Explanation

A Transformer converts text into numbers, learns relationships between words using Self-Attention, refines those representations through multiple layers, and finally predicts the next most likely word.

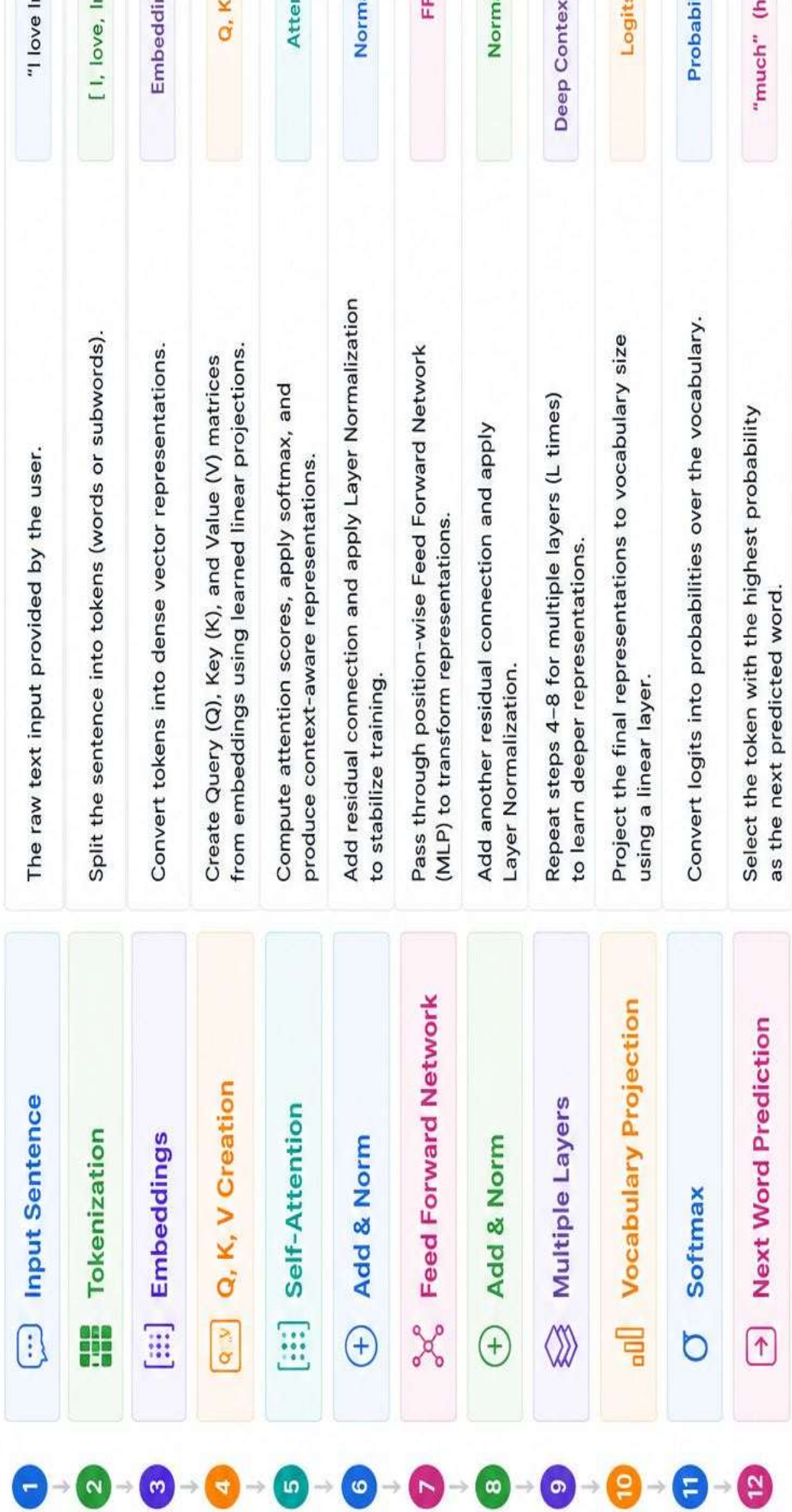


Key Takeaway

The Transformer processes the input step-by-step through multiple layers of attention and feed-forward networks to understand context and predict the next most likely word.

Overall Transformer Pipeline

Complete End-to-End Flow



Explanation

This is the complete journey of information inside a Transformer. Every modern LLM such as follows this pipeline.

Step 1 – Tokenization

The input sentence is split into smaller pieces called tokens.



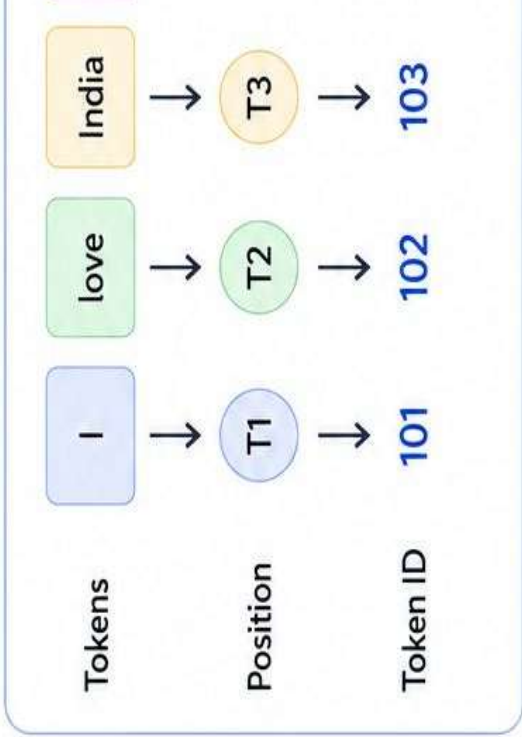
Input Sentence

I love India very much



Tokens

Position	Token	Token ID (Example)
T1	I	101
T2	love	102
T3	India	103
T4	very	104
T5	much	105



Explanation



Computers cannot understand text directly. Therefore, the sentence is broken into smaller



Each token receives a **unique ID** from the vocabulary.

After tokenization, each token is mapped to a unique numerical ID from the vocabulary.



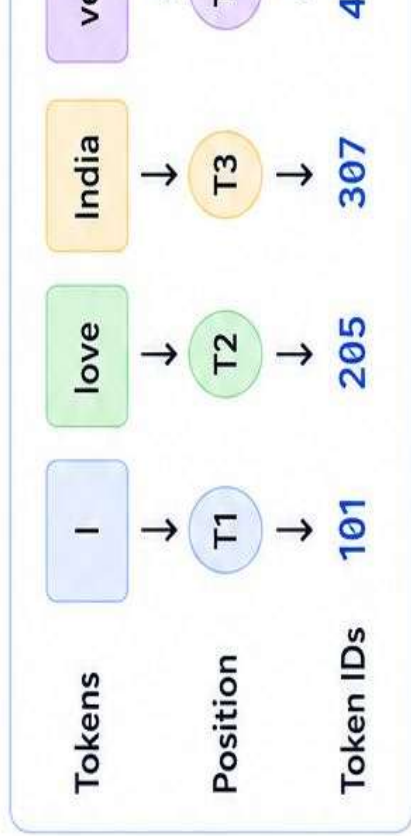
Vocabulary Mapping

Token	ID
I	101
love	205
India	307
very	412
much	528



Token Sequence (As IDs)

[101, 205, 307, 412, 528]



Explanation



The Transformer receives numerical IDs instead of words.



At this stage the model still does not know the meaning of any word.

Step 2 – Embedding Lookup

Each token ID is mapped to a dense vector from the embedding table.



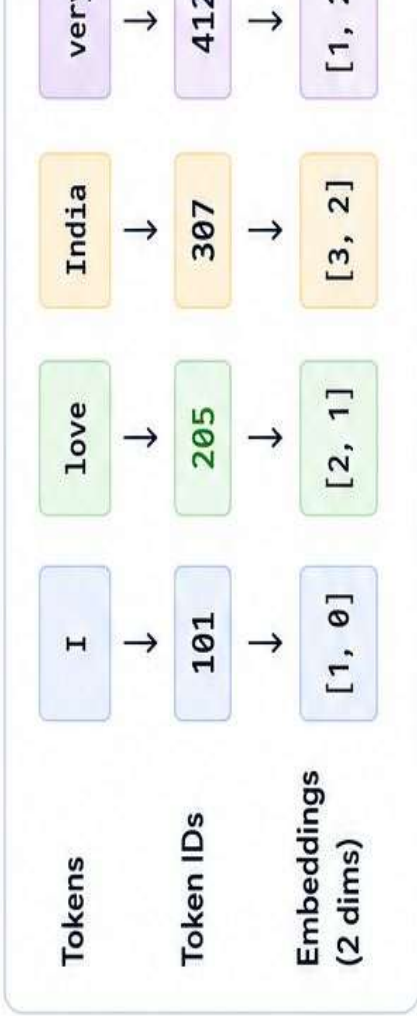
Embedding Table

Shape: 50000 × 2

Token ID	Embedding Vector (2 dims)
0	[0.12, -0.34]
1	[0.05, 0.87]
2	[-0.11, 0.23]
...	...
101	[1, 0]
205	[2, 1]
307	[3, 2]
412	[1, 2]
528	[0, 2]
...	...

[...] Retrieved Vectors (for our input tokens)

101	→	[1, 0]
205	→	[2, 1]
307	→	[3, 2]
412	→	[1, 2]
528	→	[0, 2]



Explanation

- ✓ Embeddings convert token IDs into dense vectors.
- ✓ These vectors contain semantic information learned during training.
- ✓ Words with similar meanings tend to have similar embeddings.

Embedding Matrix

The embedding matrix stacks all token vectors (embeddings) as rows.



Embedding Matrix X



Shape

Word	Token ID	Feature 1 (Dimension 1)	Feature 2 (Dimension 2)
I	101	1	0
love	205	2	1
India	307	3	2
very	412	1	2
much	528	0	2



$$X = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 3 & 2 \\ 1 & 2 \\ 0 & 2 \end{bmatrix}$$

5



Number of words (tokens)



Rows represent



Columns represent



Each number is



Explanation

- ✓ Rows represent words (or tokens) from the vocabulary.
- ✓ Columns represent learned features (dimensions) of the embeddings.
- ✓ Real Transformers use thousands of dimensions instead of only two.

Step 3 – Creating Query, Key and Value

From the embedding matrix, we create three new representations: Query (Q), Key (K) and Value (V).



Learned Matrices

W_Q

$$\begin{bmatrix} w_{q11} & w_{q12} & \dots & w_{q1d} \\ w_{q21} & w_{q22} & \dots & w_{q2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{qd1} & w_{qd2} & \dots & w_{qdd} \end{bmatrix} \quad (d \times d)$$

W_K

$$\begin{bmatrix} w_{k11} & w_{k12} & \dots & w_{k1d} \\ w_{k21} & w_{k22} & \dots & w_{k2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{kd1} & w_{kd2} & \dots & w_{kdd} \end{bmatrix} \quad (d \times d)$$

W_V

$$\begin{bmatrix} w_{v11} & w_{v12} & \dots & w_{v1d} \\ w_{v21} & w_{v22} & \dots & w_{v2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{vd1} & w_{vd2} & \dots & w_{vdd} \end{bmatrix} \quad (d \times d)$$

 These matrices are learned during training.
 d = embedding dimension.

⚡ For Simplicity

$$W_Q = W_K = W_V = I$$

(I = Identity matrix)



$$Q = XW_Q = X, \quad K = XW_K = X, \quad V = X$$

So, $Q = K = V = X$ (the embedding matrix)



Meaning of Q, K and V

Component	Meaning
Query (Q)	 What am I looking for? Used to ask: "Which tokens are relevant to me?"
Key (K)	 What information do I contain? Used to answer: "What am I about?"
Value (V)	 What information should I provide? Contains the actual information to be passed along.

From Embeddings (X) to Q, K, V



Explanation

-  Every token creates three different versions of itself: Query, Key and Value.
-  These are used in Self-Attention to determine relationships among words.

Generated Q, K, V

Using the embedding matrix X and the learned weight matrices W_Q , W_K , W_V , we generate the Query (Q), Key (K) and Value (V) matrices.

**Query Matrix**
What am I looking for?

$Q = XW_Q$

**Key Matrix**
What information do I contain?

$K = XW_K$

**Value Matrix**
What information should I provide?

$V = XW_V$

Generated Matrices (Example with 2-D Embedding)

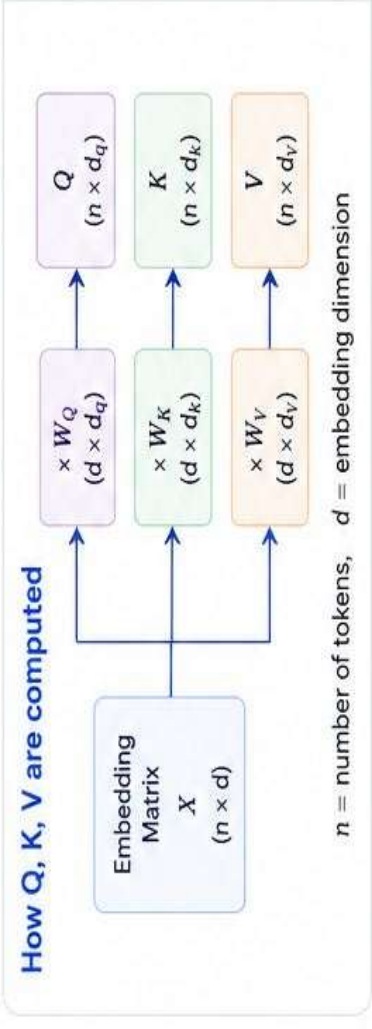
$Q = XW_Q$

$K = XW_K$

	Dim 1	Dim 2
I	1	0
love	2	1
India	3	2
very	1	2
much	0	2

	Dim 1	Dim 2
I	1	0
love	2	1
India	3	2
very	1	2
much	0	2

 In real models, these matrices have large dimensions (e.g., 768, 1024)



 **Q (Query)**
Represents what each token is looking for in other tokens

 **K (Key)**
Represents what each token contains (its identity / context)

 **V (Value)**
Represents the actual information each token provides



Explanation

- ✓ Now every word (token) has three representations: Query, Key and Value.
- ✓ These are used in Self-Attention to determine how much each word should pay attention to every other word.

Step 4 – Self Attention

Self-Attention helps each word focus on the most relevant words in the entire sentence.



Attention Formula

$$Attention = QK^T$$

(Score Matrix)

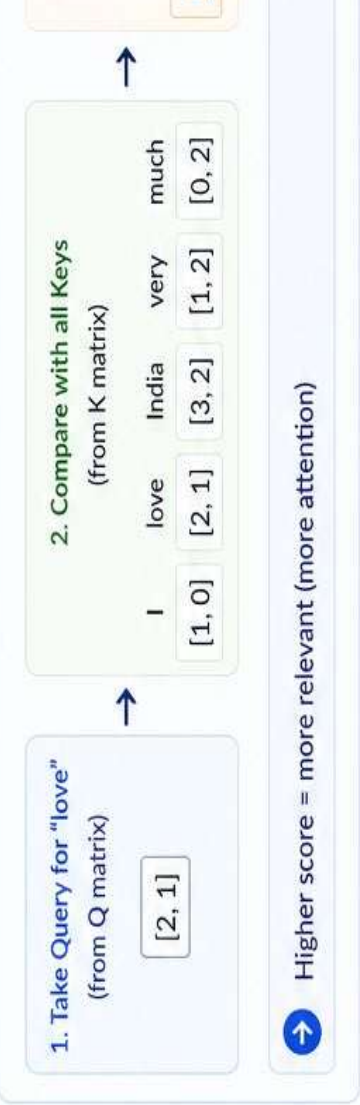


Core Idea

Each query (word) compares with all keys (words).



Example: Query = "love"



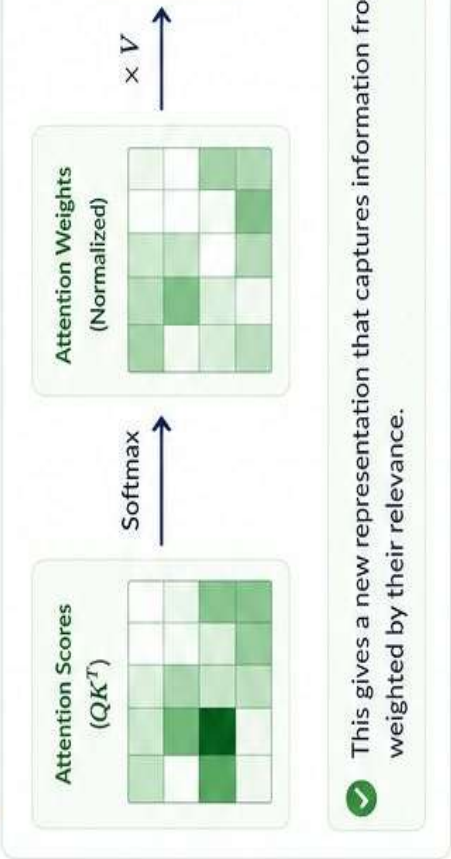
Attention Score Matrix QK^T (All Queries vs All Keys)

Queries (rows)	Keys (columns)				
	I	love	India	very	much
I	1	2	3	2	0
love	2	5	8	4	2
India	3	8	13	7	4
very	1	4	7	5	4
much	0	2	4	4	4

Each row shows how much each word (query) attends to every other word (key).



What Happens Next?



Explanation



The Transformer asks: "Which words are most relevant to this word?"



Self-Attention computes similarity scores between all pairs of words using their Query and Key vectors. These scores determine how much information each word should gather from others.

Attention Calculation for “love”

We compute how much the query for “love” attends to each key (all words in the sentence).



Computation

Query = love = [2, 1]

Key (Word)	Key Vector K	Dot Product: $Q \cdot K^T$	Score
I	[1, 0]	$(2 \times 1) + (1 \times 0) = 2$	2
love	[2, 1]	$(2 \times 2) + (1 \times 1) = 5$	5
India	[3, 2]	$(2 \times 3) + (1 \times 2) = 8$	8
very	[1, 2]	$(2 \times 1) + (1 \times 2) = 4$	4
much	[0, 2]	$(2 \times 0) + (1 \times 2) = 2$	2



Result (Scores)

$[2, 5, 8, 4, 2]$

I love India very much

These are the raw attention scores (QK^T) for the query “love”.

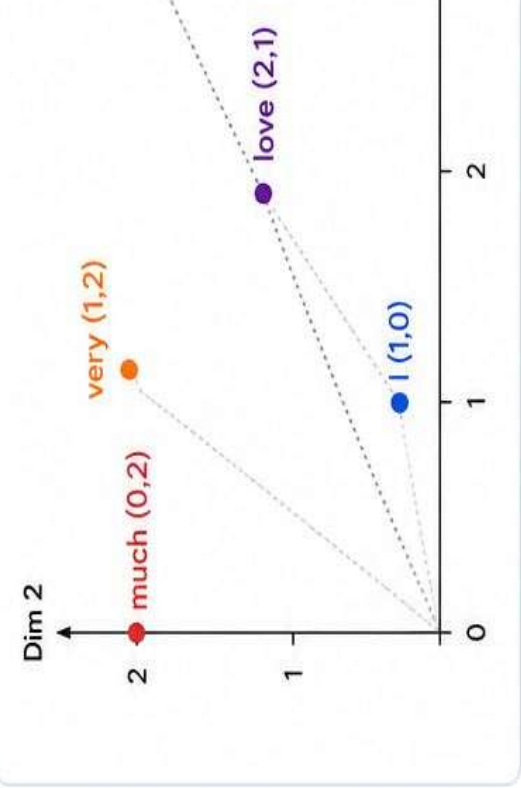


Explanation

- ✓ Self-Attention computes similarity between the query vector (for “love”) and all key vectors (all words).
- ✓ These scores will be normalized with Softmax in the next step to get attention weights.



Vector View



What It Means

- A larger score means a stronger relationship (higher similarity).
- “India” gets the highest score (8) → most relevant to the query “love”.
- “love” itself also has a high score (5) → self-relevant.
- “I” and “much” have the lowest scores (2).

Full Attention Matrix

The matrix QK^T contains attention scores between all pairs of words in the sentence.



Attention Score Matrix QK^T

		Keys (Columns)				
		I	love	India	very	much
Queries (Rows)	I	1	2	3	1	0
	love	2	5	8	4	2
	India	3	8	13	7	4
	very	1	4	7	5	4
	much	0	2	4	4	4



What Does This



Rows represent Queries
(the word that is looking)



Columns represent Keys
(the words being compared)



Each cell shows the attention score
between a query word and a key word



Higher score = stronger connection
(more relevance)



Key Takeaway

Every word now knows how strongly it is connected to every other word in the sentence.
This is the foundation of Self-Attention in Transformers.

Step 5 – Scaling

Before applying Softmax, we scale the attention scores to keep them in a stable range.



Formula

$$\frac{QK^T}{\sqrt{d_k}}$$

Scale the attention scores by the square root of the key dimension d_k .



Given

$$d_k = 2$$

$$\sqrt{2} = 1.414$$



d_k is the dimension of the Key vectors.



Why Scaling?



Without scaling, dot products are large as d_k increases.



Large values push Softmax towards 0, where it becomes very poor.



Scaling keeps the values in a stable range for better gradient flow during training.



Example

1. Raw Attention Scores (QK^T)
(from Step 4 / Slide 10)

[2, 5, 8, 4, 2]



2. Divide by $\sqrt{d_k} = 1.414$

Each value $\div 1.414$

$\left[\frac{2}{1.414}, \frac{5}{1.414}, \frac{8}{1.414}, \frac{4}{1.414}, \frac{2}{1.414} \right]$



3. Scaled Scores

[1.41, 3.54, 5.66, 2.83, 1.41]



Explanation

Scaling the scores by $\sqrt{d_k}$ prevents them from becoming too large. This helps Softmax produce well-behaved attention weights.

General Rule

Always divide QK^T by $\sqrt{d_k}$ before applying Softmax.

We convert the scaled attention scores into probabilities using the Softmax function.



Formula

$$\text{Softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Each value is exponentiated and divided by the sum of all exponentials.



What It Does

- Turns any set of values into probabilities between 0 and 1.
- Highlights larger scores and suppresses smaller ones.
- All probabilities in a row sum to 1.



Input (Scaled Scores)

[1.41, 3.54, 5.66, 2.83, 1.41]

(From Step 5 – Scaled Scores)



These are the scaled attention scores for a query (e.g., “love”) (I, love, India, very, much)



Softmax Calculation (Row-wise Example)

Word (Key)	1. Scaled Score (x_i)	2. e^{x_i}	3. Sum of e^{x_i}	4. $\text{Softmax}(x_i) = \frac{e^{x_i}}{346.42}$
I	1.41	4.10	346.42	0.01
love	3.54	34.47		0.10
India	5.66	286.77		0.82
very	2.83	16.98		0.05
much	1.41	4.10		0.01



Output (Attention Weights)

[0.01, 0.10, 0.82, 0.05, 0.01]

I love India



All probabilities sum to 1.00



“India” gets the highest probability → It receives the most attention



Explanation



Softmax converts attention scores into a probability distribution.



These probabilities determine how much attention the model pays to each word.



They will be used in the next step to compute the weighted sum of Value vectors.

Key Takeaway

Softmax ensures the output is a probability distribution on important words (less on unimportant words)

Attention Interpretation

The attention weights tell us which words the model focuses on when understanding a word.



Attention Weights (for the query word: "love")

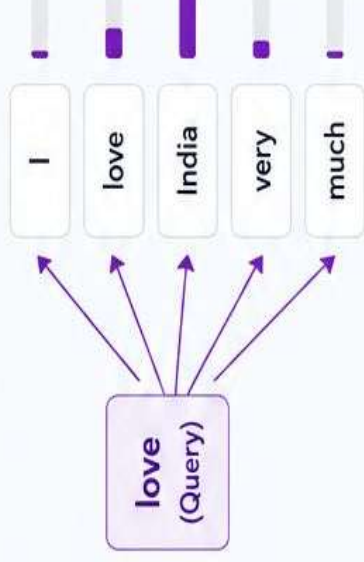
Word (Key)	Attention Weight	Interpretation
I	1%	Very little focus
love	10%	Some focus (self-relevance)
India	82%	Major focus
very	6%	Small focus
much	1%	Very little focus



What Do These Weights Mean?

When the model is processing "love", it distributes its attention across all other words in the sentence.

Word (Key)



The model mainly focuses on "India" because it is most relevant to understanding "love" in this context.



Key Takeaway



Attention weights sum to 1 (100%) and represent a probability distribution.



This is the essence of Self-Attention: the model decides how much focus to give to every other word when building its understanding.

Step 7 – Weighted Sum of Values

The model uses the attention weights to take a weighted sum of the Value vectors.



1. Values (Vectors)

Word (Key)	Value Vector V_i
I	[1, 1]
love	[2, 1]
India	[3, 2]
very	[1, 3]
much	[0, 2]



Each word has a Value vector that contains information it can provide.



2. Attention Weights (from Step 6)

These are the attention weights for the query word "love".

Word (Key)	I	love	India	very
Attention Weight	0.01	0.10	0.82	0.06



All weights sum to 1 (100%).

They tell us how much attention "love" pays to each word.



3. Weighted Combination

Compute the weighted sum of all Value vectors using the attention weights.

$$0.01 V_1 + 0.10 V_2 + 0.82 V_3 + 0.06 V_4 + 0.01 V_5$$

(I)

(love)

(India)

(very)

(much)



What this means:

Take information from each word proportional to the attention weights and add them up.



4. Step-by-Step Calculation

Word (Key)	Weight	Value Vector V_i	Weight $\times V_i$ (Element-wise)
I	0.01	[1, 1]	[0.01, 0.01]
love	0.10	[2, 1]	[0.20, 0.10]
India	0.82	[3, 2]	[2.46, 1.64]
very	0.06	[1, 3]	[0.06, 0.18]
much	0.01	[0, 2]	[0.00, 0.02]
Sum	1.00	—	[2.73, 1.95]



5. Result (New Representation)

Final weighted sum (context vector)
[2.73, 1.95]



Explanation



This new vector is a mixture of information from all words in the sentence, where more words (e.g., "India") contribute more.

This is the essence of Self-Attention: building a context-aware representation.

Residual Connection & Layer Normalization

After computing the context vector, we add a residual connection and apply Layer Normalization.



1. Residual Connection

Add the original input (query representation) to the new context vector.

$$\begin{matrix} [2, 1] \\ \text{(original)} \end{matrix} + \begin{matrix} [2.73, 1.95] \\ \text{(context)} \end{matrix} = \begin{matrix} [4.73, 2.95] \\ \text{(combined)} \end{matrix}$$



2. Layer Normalization (LayerNorm)

Normalize the combined vector to have mean 0 and variance 1.

$$\begin{matrix} [4.73, 2.95] \\ \text{(combined)} \end{matrix} \longrightarrow \begin{matrix} [1, -1] \\ \text{(normalized)} \end{matrix}$$



Why Residual Connection?

- ✓ Helps preserve the original information
- ✓ Makes it easier for the model to learn instead of replacing everything
- ✓ Improves gradient flow and helps with training deeper networks.



What Does Layer Normalization Do?

- ✓ Centers the values around 0 (zero)
- ✓ Scales them to have unit variance
- ✓ Stabilizes the output distribution
- ✓ Speeds up training and improves generalization



Key Takeaway

- ✓ Residual Connection: preserves original information and makes learning easier.
- ✓ Layer Normalization: normalizes the output for stable and efficient training.

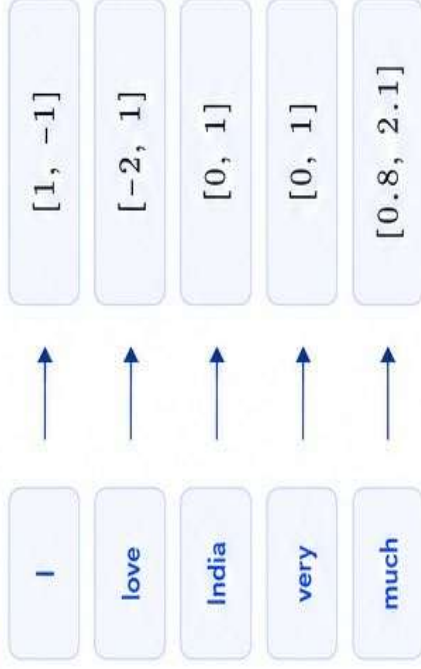
Together, these steps ensure the model **learns effectively and remains stable** as we stack more layers.

Feed Forward Network (FFN)

After Self-Attention, each position goes through a Feed Forward Network applied independently.



1. Input (After LayerNorm)



2. Feed Forward Network (per position)

A simple 2-layer MLP applied to each position:

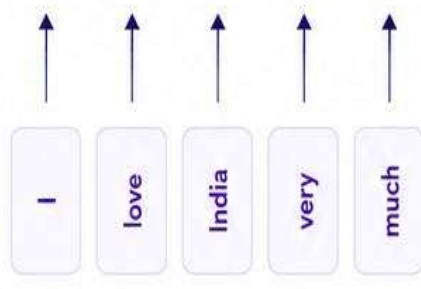


$$\text{FFN}(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2$$

- $W_1 \in \mathbb{R}^{d_{ff} \times d_{model}}$, $W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$
- $d_{ff} > d_{model}$ (expansion then projection back)
- Applied independently to each position (word)



3. Output



What Does the FFN Do?

- ✓ **Captures complex patterns:** Self-Attention mixes information from different words. FFN transforms that information with non-linear layers.
- ✓ **Adds depth to representations:** Learns richer features and higher-level abstractions.
- ✓ **Position-wise:** The same network (same weights) is applied to every position independently.



Example Intuition

Self-Attention asks:

"What words are related



FFN learns:

"Given that information, what should we extract?"



Key Takeaway

Self-Attention finds relationships between words.

Feed Forward Network learns **deeper patterns** and **transforms** those representations.

Add & Norm Again

After the Feed Forward Network, we apply another Residual Connection and Layer Normalization.



1. Residual (Add)

Add the FFN output to the input of the FFN (from previous Add & Norm).



2. Layer Normalization (LayerNorm)

Normalize the result to have mean 0 and variance 1.



Why LayerNorm Again?

- ✓ Keeps activations stable.
- ✓ Helps the model train deeper networks.
- ✓ Ensures consistent scale for the next layer.



What Happens in This Step?

- ✓ **Residual Connection:** Preserves information and makes learning easier.
- ✓ **Layer Normalization:** Stabilizes the output distribution.
- ✓ **Together:** They form the backbone of stable and efficient training in deep Transformers.



Formula Summary

Residual (Add)

$$[1, -1] + [0.8, 2.1]$$

LayerNorm

$$[1.8, 1.1] \rightarrow$$



Key Takeaway

A Transformer block follows this pattern:

Attention



Add & Norm



FFN



Add & Norm

This design helps Transformers learn while keeping training [stable](#).

Transformer Block

Attn

Add

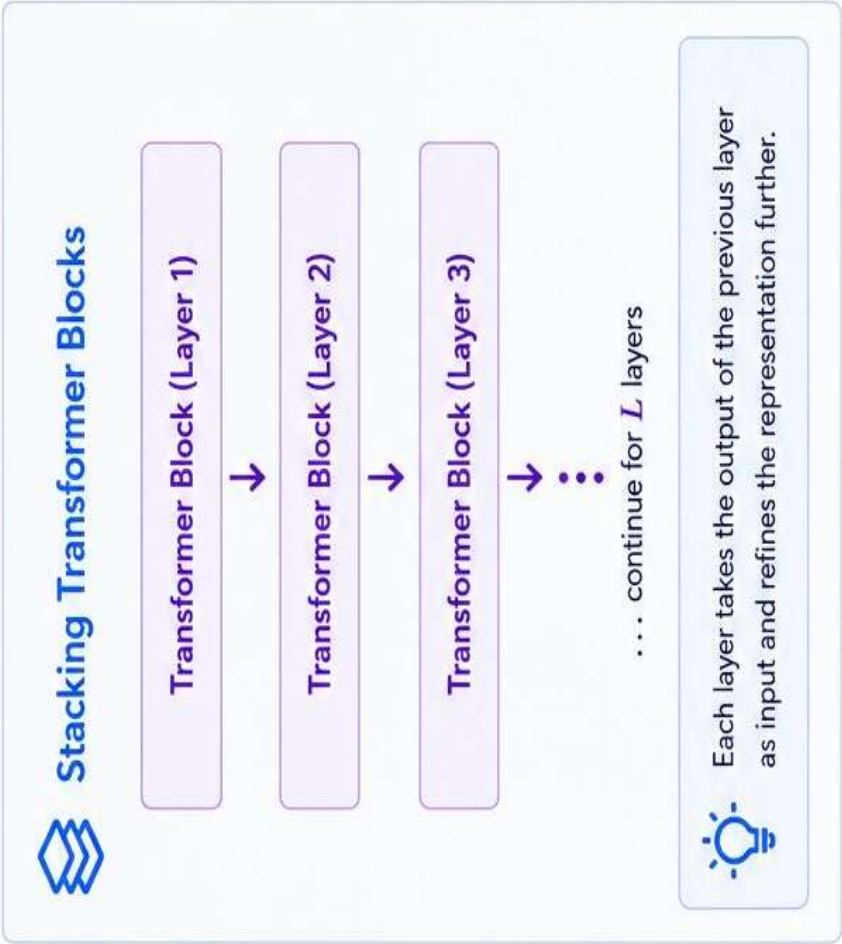
Feed Forward

Add

Output
(ready for next layer)

Multiple Transformer Layers

A Transformer stacks multiple identical blocks (layers) one after another.



Final Hidden Representation

After passing through all Transformer layers, we obtain the final hidden representation for each



Final Output

love

$$\downarrow$$
$$h_{final} = [\underbrace{3.7}_{\text{Dimension 1}}, \underbrace{8.5}_{\text{Dimension 2}}]$$



Explanation

The vector h_{final} combines contextual information gathered from the entire sentence. It is the model's **final understanding** (representation) of the word "love" in this context.



Key Takeaway

- ✓ Each token gets a final hidden vector after the last Transformer layer.
- ✓ This vector captures **rich**, **context-aware** meaning used for downstream tasks (e.g., prediction, classification, generation).

Vocabulary Projection

The final hidden vector is projected onto the vocabulary space to get scores (logits) for every possible next word

1 Output Matrix (W_{out})

A learnable matrix that maps the hidden vector to vocabulary space.

$$W_{out} \in \mathbb{R}^{V \times d_{model}}$$
$$V = \text{vocabulary size}$$
$$d_{model} = \text{hidden dimension}$$



Example Vocabulary (subset)

- India
- Pizza
- Football
- Car
- ... (and many more)

Each row in W_{out} corresponds to one vocabulary word.



Example Logits (subset)

Word		Logit (raw score)
India		15.2
Pizza		2.0
Football		2.2
Car		0.9
...		...

$$\text{logits} = h_{final} \cdot W_{out}^T$$
$$h_{final} \in \mathbb{R}^{1 \times d_{model}}$$
$$\text{logits} \in \mathbb{R}^{1 \times V}$$

2 Compute Logits

Multiply the final hidden vector h_{final} by W_{out}^T .



Explanation

These are raw scores (logits) for every word in the vocabulary.
The higher the logit, the more likely the model thinks that word is the next word.



What Happens Next?

These logits are passed through a Softmax layer to convert them into probabilities.



Formula Summary

Final hidden vector

$$h_{final} \in \mathbb{R}^{1 \times d_{model}}$$

×

Output matrix

$$W_{out}^T \in \mathbb{R}^{d_{model} \times V}$$

=

Logits

$$\in \mathbb{R}^{1 \times V}$$



Softmax

Probabilities over vocabulary

Softmax converts raw logits into probabilities over the entire vocabulary.



Softmax Function

Given a logit z_i for each vocabulary word i , Softmax computes:

$$P(i) = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}$$

$P(i)$: probability of word i

z_i : logit (raw score) for word i

V : vocabulary size



All probabilities are between 0 and 1 and sum to 1.



Explanation

- Softmax converts logits into probabilities.
- The higher the probability, the more likely the model predicts that word to be the next word.
- The word with the highest probability is the model's **predicted next word**.



Key Takeaway

From the final hidden vector $\rightarrow$ logits $\rightarrow$ Softmax $\rightarrow$ probability distribution over the vocabulary.

Final Hidden
Vector

Logits
(Raw Scores)

Softmax

Probabilities Over
Vocabulary



Predicted Probabilities (Example Subset)

Word	Logit (Raw Score)	Probability (Softmax)
India	15.2	0.9700
Pizza	2.0	0.0100
Football	2.2	0.0150
Car	0.9	0.0050
...	...	...



The probabilities across all vocabulary words add up to 1 (100%).

Next Token Prediction

The model predicts the next word in a sequence by converting context into probabilities over the vocabulary



Input (Context)



Predicted Token (Next Word)

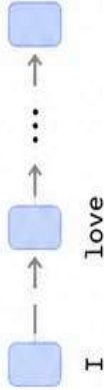
I love

India

1

Context Encoding

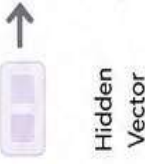
The input sequence is converted into contextual representations using Transformer layers.



2

Vocabulary Projection

The final hidden vector is projected to the vocabulary space to produce logits.



3

Softmax

Logits are converted into probabilities for every word in the vocabulary.



Probability Distribution

4

Prediction

The word with the highest probability is selected as the next token.



Highest Probability
= Predicted Token



Why this prediction?

1

Highest Attention Score

The model focused most on "India" when understanding "love" in context.



2

Strongest Contextual Representation

The hidden representation for "love" captures the meaning related to "India".



3

Highest Softmax Probability

"India" has the highest probability among all vocabulary words.

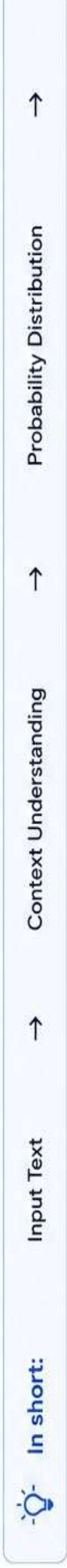
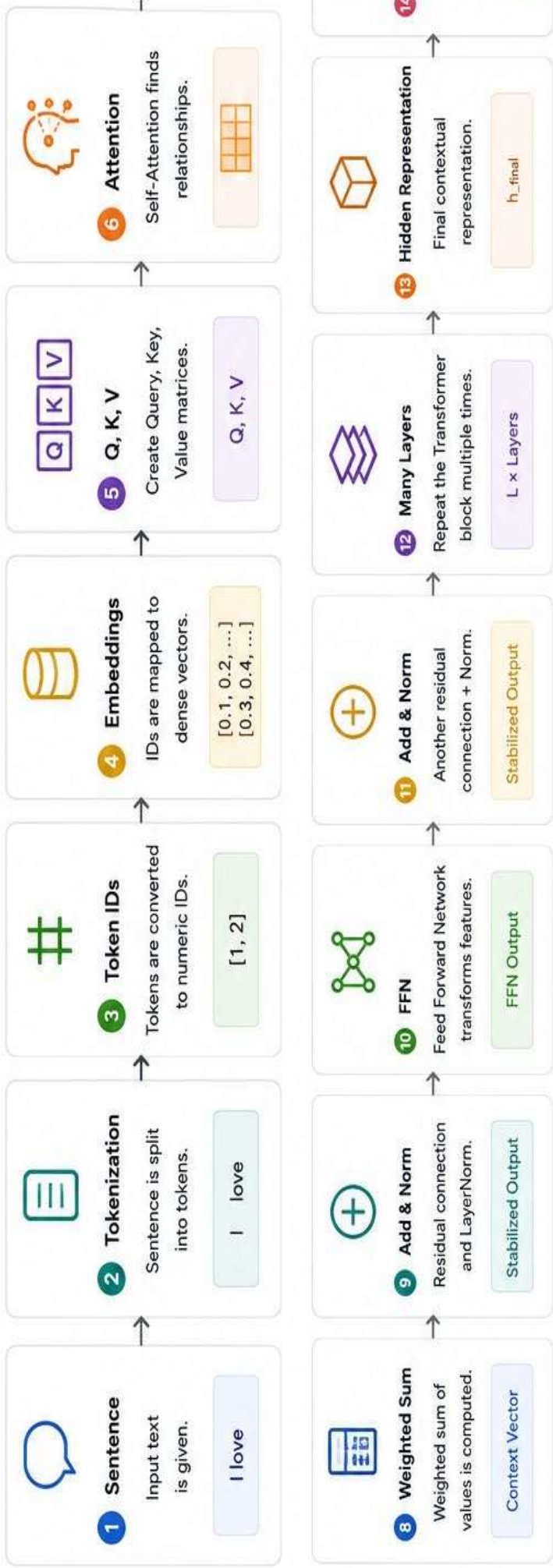


Therefore, the model predicts: **India**

This is how Transformers generate text — one token at a time

Complete Transformer Summary

Here is the end-to-end flow of how a Transformer takes a sentence and predicts the next word.



Feature	ChatGPT OpenAI	Gemini Google DeepMind	AI
 Developer	 OpenAI	 Google DeepMind	 AI
 Coding	 Excellent	 Very Good	 V
 Writing	 Excellent	 Very Good	 E
 Research	 Excellent	 Excellent	 E
 Large Documents	 Good	 Good	 E
 Google Integration	 Limited	 Excellent	 L
 Reasoning	 Excellent	 Very Good	 E

THANK YOU

